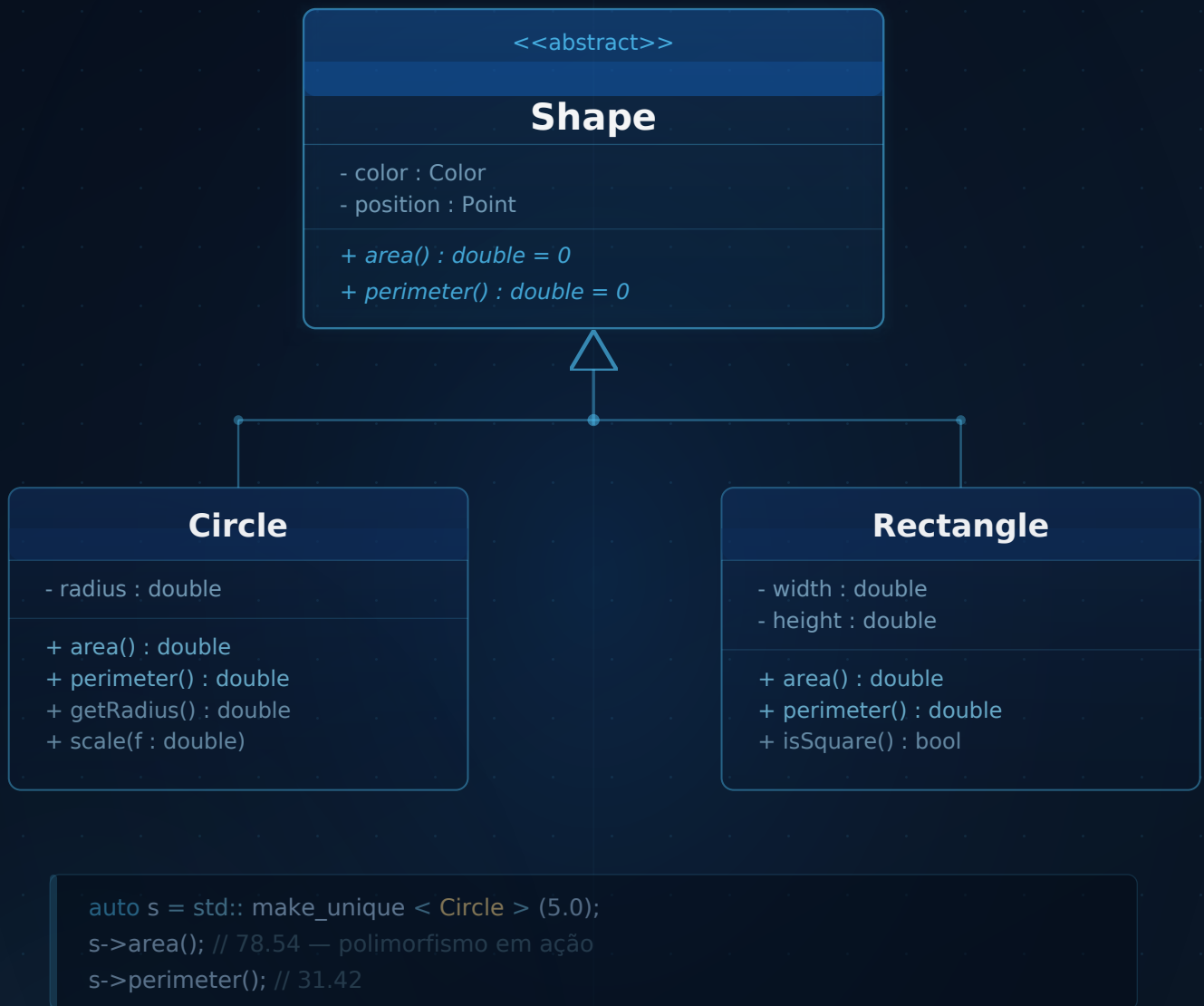




UFPB · CI · LAVID

Programação Orientada a Objetos

Do Paradigma à Implementação em C++

Carlos Eduardo C. F. Batista

Professor Associado · Departamento de Informática

1ª Edição

2026

UNIVERSIDADE FEDERAL DA PARAÍBA

Centro de Informática — Departamento de Informática

LAViD — Laboratório de Aplicações de Vídeo Digital

Programação Orientada a Objetos em C++

Fundamentos, Ciclo de Vida, Abstração e Engenharia de Software

Prof. Carlos Eduardo Coelho Freire Batista

<https://bidu.tv>

<https://github.com/dobidu>

João Pessoa, Paraíba — 2026

Carlos Eduardo C. F. Batista

Programação Orientada a Objetos em C++ — 1ª Edição © 2026

Todos os direitos reservados.

Licença: Creative Commons BY-NC-SA 4.0

Universidade Federal da Paraíba (UFPB)

Centro de Informática (CI)

Departamento de Informática

LAViD — Laboratório de Aplicações de Vídeo Digital

João Pessoa, Paraíba, Brasil

A todos os professores que um dia decidiram que ensinar valia a pena —
especialmente àqueles que continuam decidindo isso, semestre após semestre.

“Se você quer fazer uma torta de maçã do zero, primeiro precisa inventar o universo.”

Carl Sagan

Cosmos: A Personal Voyage, 1980

Prefácio

Este livro eletrônico cobre integralmente o plano de ensino da disciplina de Programação Orientada a Objetos para os cursos do Centro de Informática da Universidade Federal da Paraíba. Ela é o resultado da fusão e expansão substancial dos Volumes I e II utilizados em semestres anteriores, com adição de material novo em todos os tópicos ausentes ou rasos identificados no inventário de conteúdo.

A disciplina é organizada em torno de um projeto fio condutor chamado Sintonia — um sistema de processamento de áudio digital em C++ moderno. O projeto evolui ao longo das três unidades: de uma estrutura simples de samples e buffers (v0.1), passa por uma cadeia de efeitos polimórfica (v1.x), avança para gerenciamento genérico com templates (v2.x) e culmina em uma arquitetura SOLID com interface Qt (v3.x). Cada capítulo mostra como as decisões de projeto aprendidas afetam o Sintonia.

Convenções adotadas:

- Código: snake_case para identificadores, comentários em português, C++17 como padrão base com extensões C++20 indicadas.
- Boxes coloridos: ⚠ Cuidado (situações de perigo), 💡 Boa prática (idioma recomendado), 🤖 LLM (uso crítico de assistentes), 🎵 Sintonia (conexão com o projeto fio condutor).
- UML: diagramas de classes e sequência em Mermaid, com placeholders para inserção em ferramentas como draw.io ou PlantUML.
- Exercícios: cada capítulo termina com 4 exercícios graduados, do conceitual ao prático.

O conteúdo pressupõe certa familiaridade com programação em C (structs, ponteiros, funções) e noções básicas de Python. Não pressupõe experiência anterior com C++ ou com orientação a objetos.

O material foi compilado, atualizado e organizado com auxílio de LLMs (Large Language Models, ferramentas de inteligência artificial generativa): Claude (Anthropic), ChatGPT e Copilot (OpenAI/Microsoft) para texto e código; Gemini (Google) e Qwen (Alibaba) para imagens e diagramas. O conteúdo técnico, a curadoria e a responsabilidade editorial são inteiramente do autor.

Prof. Carlos Eduardo Coelho Freire Batista
João Pessoa, Paraíba, 2026

Sumário

Prefácio	5
Sumário	6
Capítulo 1 — Da Programação Procedural à Orientação a Objetos	13
1.1 Gerenciamento de Complexidade em Software	13
1.2 Limitações do Modelo Procedural	14
1.3 Os Quatro Pilares da POO	14
Encapsulamento	14
Herança	14
Polimorfismo	15
Abstração	15
1.4 Exemplo Comparativo: Sistema de Alunos em C vs. C++	15
Versão C (Procedural)	15
Versão C++ (Orientado a Objetos)	16
1.5 Comparando: C Procedural vs. C++ OO	17
Exercícios Propostos	17
Capítulo 2 — Fundamentos de C++ para POO	18
2.1 Entrada e Saída: de printf/scanf a cout/cin	18
2.2 Strings: de char[] a std::string	18
2.3 Ponteiros vs. Referências	19
2.4 C++ Moderno Essencial	20
auto — Dedução de Tipo	20
nullptr — Ponteiro Nulo Seguro	21
Inicialização Uniforme com {}	21
Exercícios Propostos	21
Capítulo 3 — Classificação de Linguagens Baseadas em Objetos e Sistemas de Tipos	23
3.1 Baseadas em Objetos vs. Orientadas a Objetos	23
3.2 Sistemas de Tipos: os Eixos Fundamentais	23
Eixo 1 — Tipagem Estática vs. Dinâmica	23
Eixo 2 — Tipagem Forte vs. Fraca	23
Eixo 3 — Single Dispatch vs. Multiple Dispatch	24
3.3 Mecanismos de Reuso: Herança, Composição e Traits	24
3.4 Posicionando as Linguagens	24
Exercícios Propostos	25
Capítulo 4 — Infraestrutura do Programador C++	26
4.1 O Modelo de Compilação de C++	26
4.2 CMake	27

Estrutura mínima de projeto	27
4.3 GDB — Depuração Básica	28
4.4 Sanitizers — Detecção de Erros em Runtime	28
AddressSanitizer (ASan)	28
UndefinedBehaviorSanitizer (UBSan)	29
4.5 Transição de Python/C para C++	29
Exercícios Propostos	29
Capítulo 5 — Git e GitHub como Ambiente de Trabalho	31
5.1 Por que Versionar Código?	31
5.2 Conceitos Fundamentais	31
5.3 Fluxo de Trabalho Individual	31
5.4 Branches e Merge	32
5.5 GitHub — Colaboração e Entrega	32
5.6 .gitignore para Projetos C++	33
Exercícios Propostos	33
Capítulo 6 — Assistentes de Programação Baseados em LLM como Copiloto	35
6.1 O que São LLMs e Como Funcionam	35
6.2 Prompts Eficazes para POO	35
Estrutura de um bom prompt de programação	35
6.3 Prompts por Contexto de Desenvolvimento OO	36
Modelagem — Esboço de Classes	36
Refatoração — Aplicando SOLID	36
Geração de Testes	36
6.4 Análise Crítica de Código Gerado por LLMs	36
6.5 Limites, Riscos e Responsabilidades	37
Exercícios Propostos	38
Capítulo 7 — UML Leve como Ferramenta de Projeto	39
7.1 Por que UML? A Modelagem Antes do Código	39
7.2 Diagrama de Classes — Notação Essencial	39
7.3 Relacionamentos entre Classes	40
7.4 Diagrama de Sequência — Interação entre Objetos	41
7.5 Fluxo de Trabalho com UML Leve	41
Exercícios Propostos	42
Capítulo 8 — Classes e Objetos em C++	43
8.1 Declaração e Definição de Classes	43
8.2 Modificadores de Acesso	44
8.3 O Ponteiro this	44
8.4 Const-Correctness	45

8.5 Membros Estáticos (static)	45
Exercícios Propostos	46
Capítulo 9 — Ciclo de Vida de Objetos: Construtores, Destrutores e RAI	47
9.1 O que é o Ciclo de Vida de um Objeto?	47
9.2 Tipos de Construtores	47
Construtor Padrão	47
Construtor Parametrizado	48
Construtor de Cópia	48
9.3 Listas de Inicialização — Por que Usar Sempre	48
9.4 Destrutores e a Ordem de Destruição	49
9.5 RAI — Resource Acquisition Is Initialization	49
Exercícios Propostos	50
Capítulo 10 — RAI e a Regra dos Zero/Três/Cinco	53
10.1 O que o Compilador Gera Automaticamente	53
10.2 Regra do Zero	53
10.3 Regra dos Três (Pré-C++11)	54
10.4 Regra dos Cinco (C++11 em diante)	55
10.5 Semântica de Valor vs. Referência	56
Exercícios Propostos	56
Capítulo 11 — Smart Pointers	57
11.1 Por que Smart Pointers?	57
11.2 unique_ptr — Propriedade Exclusiva	57
11.3 shared_ptr — Propriedade Compartilhada	58
11.4 weak_ptr — Quebrando Ciclos	58
11.5 Regra de Escolha	58
Exercícios Propostos	59
Capítulo 12 — Semântica de Movimento	60
12.1 O Problema: Cópias Desnecessárias	60
12.2 lvalues e rvalues	60
12.3 std::move e Referências a rvalue (T&&)	60
Exercícios Propostos	61
Capítulo 13 — Sobrecarga de Operadores	62
13.1 Por que Sobrecarregar Operadores?	62
13.2 Regras de Ouro	62
13.3 Exemplo: sample com Operadores	62
Exercícios Propostos	63
Capítulo 14 — Testes Automatizados com Catch2	64
14.1 Por que Testes Automatizados?	64

14.2 Configurando Catch2 com CMake	64
14.3 Primeiro Arquivo de Teste	64
14.4 Testando Comportamentos de Exceção e Edge Cases	65
Exercícios Propostos	66
Capítulo 15 — Herança Simples em C++	67
15.1 Herança como Modelagem de Domínio	67
15.2 Sintaxe e Tipos de Herança em C++	67
15.3 Hierarquia de Efeitos — Sintonia v1.0	67
15.4 Construtores e Destrutores na Hierarquia	69
Exercícios Propostos	69
Capítulo 16 — Herança Múltipla e o Problema do Diamante	71
16.1 Herança Múltipla: Capacidades e Riscos	71
16.2 O Problema do Diamante	71
16.3 Resolvendo com Herança Virtual	72
16.4 Contraste com Java/C#	73
Exercícios Propostos	73
Capítulo 17 — Funções Virtuais e Classes Abstratas	75
17.1 A Necessidade de Funções Virtuais	75
17.2 override e final	75
17.3 Funções Virtuais Puras e Classes Abstratas	76
17.4 Mecanismo Interno: vtable e vptr	76
Exercícios Propostos	78
Capítulo 18 — Polimorfismo Dinâmico, RTTI e dynamic_cast	79
18.1 Polimorfismo Dinâmico em Ação	79
18.2 dynamic_cast — Downcasting Seguro	79
18.3 RTTI — typeid e type_info	80
Exercícios Propostos	80
Capítulo 19 — Templates e Polimorfismo Estático	83
19.1 O que São Templates?	83
19.2 Templates de Função	83
19.3 Templates de Classe — Buffer<T>	84
19.4 Polimorfismo Estático vs. Dinâmico	84
19.5 CRTP — Curiously Recurring Template Pattern	85
Exercícios Propostos	85
Capítulo 20 — Concepts (C++20) e Ranges	87
20.1 O Problema dos Templates sem Restrições	87
20.2 Ranges — Programação Funcional sobre Sequências	87
Exercícios Propostos	88

Capítulo 21 — Tratamento de Erros	89
21.1 Exceções em C++	89
21.2 std::optional — Resultado Ausente sem Exceção	89
21.3 std::variant — Resultado ou Erro	90
Exercícios Propostos	90
Capítulo 22 — STL Panorâmica	92
22.1 Contêineres Essenciais	92
22.2 Algoritmos STL	92
Capítulo 23 — Concorrência em C++ — Panorâmica	94
23.1 std::thread — Criação e Join	94
23.2 std::mutex e Race Conditions	94
Capítulo 24 — Serialização	95
24.1 Estratégias de Serialização	95
24.2 Serialização JSON com nlohmann/json	95
Capítulo 25 — Princípios SOLID	96
25.1 S — Responsabilidade Única (SRP)	96
25.2 O — Aberto/Fechado (OCP)	96
25.3 L — Substituição de Liskov (LSP)	96
25.4 I — Segregação de Interfaces (ISP)	97
25.5 D — Inversão de Dependência (DIP)	97
Exercícios Propostos	98
Capítulo 26 — Padrões de Projeto Canônicos em C++ Moderno	99
26.1 Strategy — Encapsular Algoritmos	99
26.2 Observer — Notificação de Eventos	99
26.3 Factory Method — Criação Polimórfica	100
26.4 Decorator — Efeitos Encadeados	100
26.5 Composite — Hierarquia de Efeitos	101
26.6 Singleton — Uso Correto e Abusos	101
Exercícios Propostos	102
Capítulo 27 — Qt na Prática e LLMs no Ciclo OO	103
27.1 Arquitetura do Qt	103
27.2 Separação Domínio/Apresentação com Qt	104
27.3 LLMs no Ciclo OO — Aprofundamento	104
Exercícios Propostos	105
Glossário	106
Referências Bibliográficas	109
Referências Básicas	109
Referências Complementares	109

UNIDADE I

Fundamentos do Paradigma OO e Entrada em C++ Moderno

Do C estruturado ao C++ orientado a objetos - conceitos, infraestrutura e primeiras classes

Capítulo 1 — Da Programação Procedural à Orientação a Objetos

🎯 Objetivos deste capítulo

- ▶ Reconhecer as limitações do modelo procedural em sistemas de software de grande escala.
- ▶ Compreender os quatro pilares da POO: encapsulamento, herança, polimorfismo e abstração.
- ▶ Contrastar implementações equivalentes em C procedural e C++ orientado a objetos.
- ▶ Identificar quando a orientação a objetos representa ganho real de expressividade.

1.1 Gerenciamento de Complexidade em Software

À medida que sistemas crescem, o principal desafio do programador deixa de ser "fazer funcionar" e passa a ser "entender, modificar e testar sem quebrar". Pesquisas empíricas mostram que programadores passam mais de 70% do tempo lendo código — não escrevendo. A Programação Orientada a Objetos (POO) surgiu como resposta direta a esse problema: ela organiza o código (Figura 1) de forma que cada pedaço cuide da sua própria responsabilidade, reduza as dependências entre partes e torne os efeitos de uma mudança mais previsíveis.



Figura 1. Uma visão panorâmica de um laboratório de tecnologia, contrastando o caos procedural (C) com a ordem orientada a objetos (C++).

Os paradigmas de programação são estilos fundamentalmente distintos de organizar um programa. Os mais relevantes para esta disciplina são:

- **Paradigma procedural/estruturado (C, Pascal):** o programa é uma sequência de funções que operam sobre dados globais ou passados por parâmetro.
- **Paradigma orientado a objetos (C++, Java, Python, C#):** dados e operações são agrupados em objetos que se comunicam por mensagens.
- **Paradigma funcional (Haskell, Erlang, partes de Python e C++):** computação como aplicação de funções puras sem efeitos colaterais.
- **Paradigma genérico (templates em C++, generics em Java):** algoritmos parametrizados por tipo, avaliados em tempo de compilação.

C++ é multiparadigma: suporta POO, programação genérica (templates), programação procedural e aspectos funcionais. Isso o torna especialmente poderoso — e especialmente exigente do programador.

1.2 Limitações do Modelo Procedural

Para entender por que a POO foi criada, é preciso sentir as dores reais do modelo procedural. Considere um sistema simples de gestão de alunos em C:

Problema 1 — Falta de encapsulamento em C

```
// Em C: dados e funções vivem em mundos separados
struct student {
    int id;
    char name[50];
    float grade1, grade2, grade3;
};

// Qualquer código pode fazer isso — não há proteção:
struct student s;
s.grade1 = -50.0f;    // nota inválida, mas compila
s.id     = 0;         // id inválido, mas compila
```

Problema 2 — Dados e funções desconectados

```
// Dados e funções desconectados — erros conceituais não são pegos:
void print_student(struct student* s) { /* ... */ }

struct classroom cls;
print_student(&cls); // Erro conceitual — compilador pode aceitar!
```

Problema 3 — Gerenciamento manual e frágil

```
// Inicialização manual obrigatória — fácil de esquecer:
struct student s;           // dados INDEFINIDOS
init_student(&s, 123, "Ana"); // obrigatório — mas sem garantia
```

Cuidado:

Em C, nada impede que outro módulo acesse e modifique diretamente os campos de uma struct. Projetos grandes com centenas de structs e milhares de funções se tornam rapidamente impossíveis de manter — qualquer mudança pode quebrar código em qualquer lugar.

1.3 Os Quatro Pilares da POO

A POO resolve os problemas acima por meio de quatro mecanismos fundamentais:

Encapsulamento

Dados e operações são agrupados em uma unidade chamada classe. O acesso aos dados é controlado: o que é public pode ser acessado de fora; o que é private só pode ser acessado pelos próprios métodos da classe. Isso garante que os dados estejam sempre em estado válido.

Herança

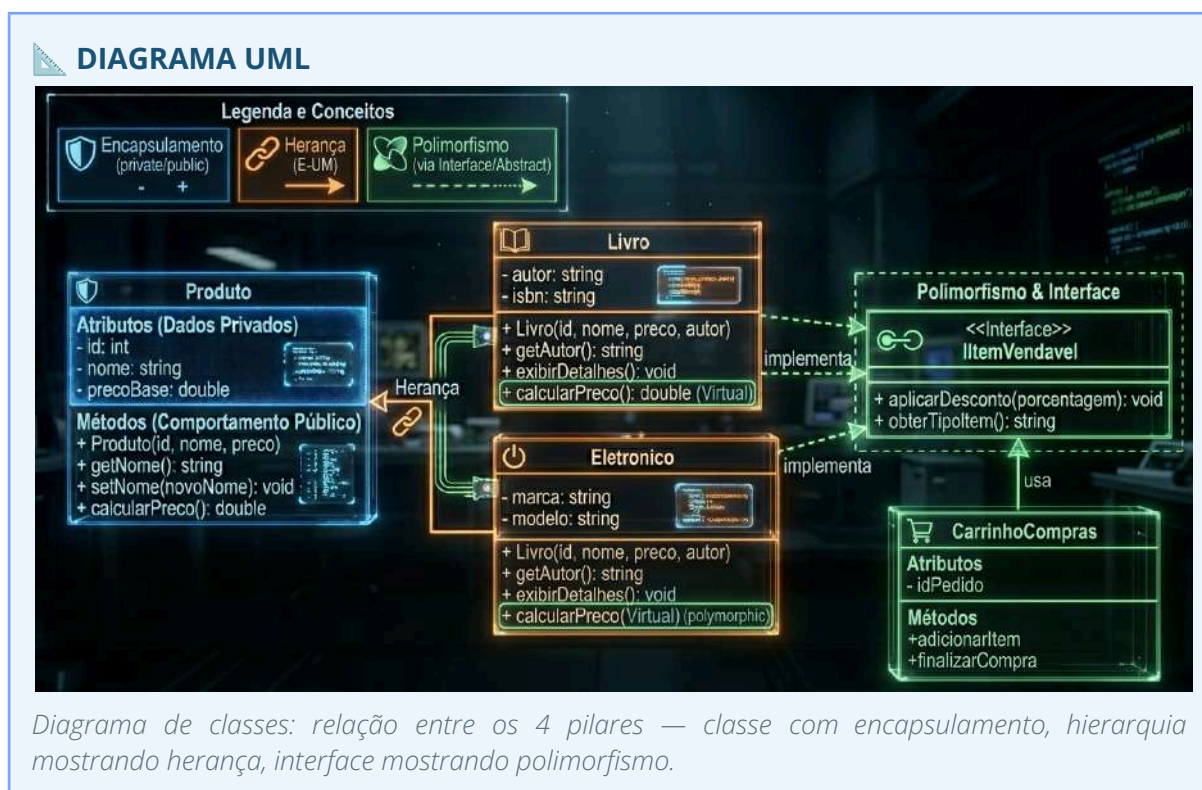
Novas classes são construídas a partir de classes existentes, herdando seus atributos e métodos. A classe derivada pode especializar o comportamento sem repetir código. Modela a relação "é um" (is-a): um Cachorro é um Animal.

Polimorfismo

Objetos de tipos diferentes respondem à mesma mensagem de maneiras distintas. Um método virtual process() em uma classe Effect pode se comportar diferentemente em Reverb, Delay ou Distortion — o código chamador não precisa saber qual tipo concreto está usando.

Abstração

O programador expõe apenas o que é essencial, ocultando detalhes de implementação. Uma classe Stack oferece push() e pop() sem revelar se usa array ou lista ligada internamente.



1.4 Exemplo Comparativo: Sistema de Alunos em C vs. C++

Versão C (Procedural)

student.c — Implementação procedural em C

```
#include <stdio.h>
#include <string.h>

/* Estrutura: dados sem proteção */
struct student {
    int id;
    char name[50];
    float grade1, grade2, grade3;
};
```

```

void init_student(struct student* s, int id, const char* name) {
    s->id = id;
    s->grade1 = s->grade2 = s->grade3 = 0.0f;
    strncpy(s->name, name, 49);
    s->name[49] = '\0';
}

float avg(const struct student* s) {
    return (s->grade1 + s->grade2 + s->grade3) / 3.0f;
}

void print_student(const struct student* s) {
    printf("ID: %d Nome: %s Media: %.2f\n", s->id, s->name, avg(s));
}

int main(void) {
    struct student s;
    init_student(&s, 1001, "Ana Lima");
    s.grade1 = 8.5f; s.grade2 = 7.0f; s.grade3 = 9.0f;
    print_student(&s);
    return 0;
}

```

Versão C++ (Orientado a Objetos)

student.cpp — Implementação orientada a objetos em C++

```

#include <iostream>
#include <string>
#include <stdexcept>

/* Classe: dados e métodos juntos, com proteção */
class student {
public:
    student(int id, std::string name)
        : id_{id}, name_{std::move(name)},
          grade1_{0.0f}, grade2_{0.0f}, grade3_{0.0f} {}

    /* Validação automática dentro do setter */
    void set_grades(float g1, float g2, float g3) {
        auto validate = [](float g) {
            if (g < 0.0f || g > 10.0f)
                throw std::domain_error("nota fora do intervalo [0, 10]");
            return g;
        };
        grade1_ = validate(g1);
        grade2_ = validate(g2);
        grade3_ = validate(g3);
    }

    float avg() const { return (grade1_ + grade2_ + grade3_) / 3.0f; }
    const std::string& name() const { return name_; }
    int id() const { return id_; }

    void print() const {
        std::cout << "ID: " << id_ << " Nome: " << name_
                    << " Media: " << avg() << "\n";
    }

private:
    int id_;
    std::string name_;
    float grade1_, grade2_, grade3_;
};

```



```
int main() {
    student s{1001, "Ana Lima"};
    s.set_grades(8.5f, 7.0f, 9.0f);
    s.print();
    return 0;
}
```

Boa prática:

Observe: (1) dados são privados — ninguém pode atribuir nota inválida de fora; (2) o construtor garante que o objeto nasce em estado válido; (3) o main() não usa ponteiros e não precisa inicializar manualmente.

1.5 Comparando: C Procedural vs. C++ OO

Aspecto	C Procedural	C++ Orientado a Objetos
Organização	struct + funções separadas	class reúne dados e métodos
Inicialização	init_student(&s, ...) — manual	Construtor automático: student s{id, nome}
Proteção	Acesso direto a qualquer campo	Campos private — acesso via métodos
Validação	Manual em cada função	Automática dentro dos métodos
Reutilização	Duplicação de código	Herança: class grad_student : public student
Strings	char[] + strcpy/strcmp	std::string com operadores naturais
Coleções	Arrays de tamanho fixo	std::vector — dinâmico e seguro

Exercícios Propostos

1. Identifique três limitações reais do sistema de alunos em C mostrado na seção 1.4. Para cada limitação, descreva um cenário concreto em que ela causaria um bug difícil de encontrar.
2. Reescreva a versão C adicionando uma função set_grade() que valide o valor. Por que isso não resolve o problema de encapsulamento de forma satisfatória?
3. Adicione à classe student em C++ um método bool passed() const que retorne true se a média for ≥ 5.0 . Escreva um teste simples no main() para verificar casos de aprovação e reprovação.
4. Pesquise e descreva com suas palavras como o Python implementa encapsulamento (convenção de underscore) em comparação com C++ (modificadores de acesso). Qual abordagem oferece garantias mais fortes?

Capítulo 2 — Fundamentos de C++ para POO

🎯 Objetivos deste capítulo

- ▶ Dominar a entrada e saída com streams (cin/cout) e suas diferenças em relação a printf/scanf.
- ▶ Utilizar std::string no lugar de arrays de char.
- ▶ Compreender ponteiros e referências e saber quando usar cada um.
- ▶ Aplicar conceitos essenciais de C++ moderno: auto, nullptr, inicialização uniforme.

2.1 Entrada e Saída: de printf/scanf a cout/cin

A transição de C para C++ é imediatamente visível no sistema de I/O. Em C, printf() e scanf() usam especificadores de formato (%d, %f, %s) que o compilador não verifica — um erro de tipo é comportamento indefinido silencioso. Em C++, os operadores << e >> com streams detectam o tipo automaticamente.

Comparação: printf vs cout

```
/* C – propenso a erros de tipo */
int n = 42;
printf("%f\n", n);          /* UB silencioso: especificador errado */

// C++ – seguro em tipos
int n = 42;
std::cout << n << "\n"; // sempre correto: operator<< sobrecarregado para int
```

Entrada robusta em C++

```
#include <iostream>
#include <string>
#include <limits>

int main() {
    int age;
    std::string name;

    std::cout << "Nome: ";
    std::getline(std::cin, name);    // lê linha inteira, incluindo espaços

    std::cout << "Idade: ";
    std::cin >> age;
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

    std::cout << "Ola, " << name << "! Voce tem " << age << " anos.\n";
    return 0;
}
```

⚠ Cuidado:

Misturar cin >> com getline() sem ignorar o newline residual causa bugs clássicos: o getline() lê uma string vazia porque encontra imediatamente o \n deixado pelo >>. Use cin.ignore() entre eles.

2.2 Strings: de char[] a std::string

Arrays de char em C exigem gerenciamento manual de tamanho, strcpy(), strcat() e são a fonte histórica de buffer overflows. std::string gerencia memória automaticamente, tem operadores naturais e métodos seguros.

std::string — operações essenciais

```
#include <iostream>
#include <string>
#include <algorithm>

int main() {
    std::string s1 = "ola";
    std::string s2 = "mundo";

    std::string s3 = s1 + ", " + s2 + "!"; // concatenação com +

    std::cout << s3 << "\n";                // "ola, mundo!"
    std::cout << s3.size() << "\n";          // 11
    std::cout << s3.substr(4, 5) << "\n";    // "mundo"

    /* busca */
    if (s3.find("mundo") != std::string::npos)
        std::cout << "encontrado\n";

    /* maiúsculas */
    std::string upper = s3;
    std::transform(upper.begin(), upper.end(), upper.begin(), ::toupper);
    std::cout << upper << "\n";              // "OLA, MUNDO!"
    return 0;
}
```

2.3 Ponteiros vs. Referências

Ponteiros e referências são os dois mecanismos de C++ para acessar um objeto indiretamente (Figura 2).



Figura 2. Uma exploração visual de referências e ponteiros, ilustrando como as referências são aliases constantes enquanto os ponteiros podem apontar para diferentes objetos ou até mesmo para o void (nullptr).

Entender a diferença é fundamental para passar parâmetros corretamente e para compreender smart pointers mais adiante.

Característica	Ponteiro (T*)	Referência (T&)
Pode ser nulo?	Sim (nullptr)	Não — deve sempre referenciar um objeto válido
Pode ser reassinalado?	Sim	Não — sempre aponta para o mesmo objeto
Sintaxe de acesso	p->campo ou (*p).campo	ref.campo (igual ao objeto)
Aritmética?	Sim (p++, p+n)	Não
Uso típico	Propriedade optativa, arrays, polimorfismo	Passagem por referência em funções, operadores

As três formas de passagem de parâmetros

```
#include <iostream>
#include <string>

// Passagem por valor: cópia — modificações não afetam o original
void by_value(std::string s) { s += " [modificado]"; }

// Passagem por referência: eficiente e modifica o original
void by_ref(std::string& s) { s += " [modificado]"; }

// Passagem por referência const: eficiente, só leitura
void by_cref(const std::string& s) { std::cout << s << "\n"; }

int main() {
    std::string msg = "ola";
    by_value(msg); std::cout << msg << "\n"; // "ola" — não mudou
    by_ref(msg);   std::cout << msg << "\n"; // "ola [modificado]"
    by_cref(msg);  // lê sem copiar
    return 0;
}
```

Boa prática:

Regra geral de passagem de parâmetros em C++ moderno:

- (1) tipos pequenos (int, float, char) — por valor;
- (2) objetos que serão modificados — por referência (T&);
- (3) objetos somente leitura — por const& (T const&);
- (4) quando o parâmetro pode ser nulo — por ponteiro (T*).

2.4 C++ Moderno Essencial

auto — Dedução de Tipo

auto instrui o compilador a deduzir o tipo a partir da expressão de inicialização. Reduz verbosidade sem perder segurança de tipos — o tipo ainda é verificado em tempo de compilação.

```
auto e range-based for
#include <vector>
```

```
#include <map>
#include <string>

int main() {
    auto x = 42;                // int
    auto pi = 3.14159;          // double
    auto name = std::string{"Ana"}; // std::string

    std::vector<int> nums = {1, 2, 3, 4, 5};

    // sem auto: std::vector<int>::iterator it = nums.begin();
    for (auto it = nums.begin(); it != nums.end(); ++it)
        std::cout << *it << " ";

    // range-based for — a forma mais idiomática:
    for (auto n : nums) std::cout << n << " ";
    for (const auto& n : nums) std::cout << n << " "; // sem cópia
    return 0;
}
```

nullptr — Ponteiro Nulo Seguro

nullptr é um literal de tipo std::nullptr_t, seguro para sobrecarga de funções. NULL é simplesmente 0 (inteiro) e pode causar ambiguidades em código com sobrecarga.

nullptr vs NULL — por que importa

```
void f(int n) { std::cout << "inteiro\n"; }
void f(int* p) { std::cout << "ponteiro\n"; }

f(NULL);    // ambíguo em C++ — pode chamar f(int)!
f(nullptr); // sem ambiguidade — chama f(int*)
```

Inicialização Uniforme com {}

C++11 introduziu a sintaxe de chaves para inicializar qualquer objeto de forma consistente. Além de unificar a sintaxe, a inicialização com chaves proíbe narrowing conversions em tempo de compilação.

Inicialização uniforme — {}

```
int a{42};           // OK
int b{3.7};          // ERRO de compilação: narrowing (double -> int)

std::vector<int> v{1, 2, 3, 4, 5}; // lista de inicializadores
std::string s{"ola"};

// Útil para objetos complexos:
struct point { int x, y; };
point p{10, 20};
```

Exercícios Propostos

1. Escreva um programa que leia nome e três notas de um aluno usando std::getline e cin >>, trate o problema do newline residual e imprima a média formatada com duas casas decimais usando std::fixed e std::setprecision.

- 2.** Implemente uma função `void reverse_words(std::string& s)` que inverta a ordem das palavras em uma string sem usar contêineres auxiliares. Dica: use `std::reverse` de `<algorithm>`.
- 3.** Escreva duas versões de uma função `swap` que troque os valores de dois inteiros: uma com ponteiros (estilo C) e outra com referências (estilo C++). Qual é mais segura e por quê?
- 4.** Explique com um exemplo concreto por que `NULL` pode ser ambíguo em C++ ao usar sobrecarga de funções e como `nullptr` resolve esse problema.

Capítulo 3 — Classificação de Linguagens Baseadas em Objetos e Sistemas de Tipos

🎯 Objetivos deste capítulo

- ▶ Distinguir linguagens "baseadas em objetos" de linguagens "orientadas a objetos".
- ▶ Compreender os eixos de classificação: tipagem estática vs. dinâmica, forte vs. fraca, herança vs. composição vs. traits.
- ▶ Posicionar C++, Python, Java, JavaScript e Rust nessa taxonomia.
- ▶ Entender o sistema de tipos de C++ e suas implicações práticas.

3.1 Baseadas em Objetos vs. Orientadas a Objetos

Nem toda linguagem que usa a palavra "objeto" é verdadeiramente orientada a objetos. A distinção clássica de Wegner (1987) estabelece:

Categoria	Características	Exemplos
Baseadas em objetos	Tem objetos e encapsulamento, mas sem herança nem polimorfismo	Ada 83, primeiras versões do Visual Basic
Orientadas a objetos	Objetos + herança + polimorfismo (despacho dinâmico)	C++, Java, Python, Ruby, Smalltalk
Multiparadigma	OO + funcional + genérico + procedural	C++, Scala, Kotlin, Rust (sem herança clássica)

C++ é a linguagem multiparadigma por excelência: você pode escrever C puro, POO com herança e polimorfismo, código genérico com templates, código funcional com lambdas e algoritmos da STL — tudo no mesmo projeto.

3.2 Sistemas de Tipos: os Eixos Fundamentais

Eixo 1 — Tipagem Estática vs. Dinâmica

Em linguagens de tipagem estática (C++, Java, Rust), os tipos são verificados em tempo de compilação — erros de tipo são detectados antes de rodar o programa. Em linguagens de tipagem dinâmica (Python, JavaScript, Ruby), os tipos são verificados em tempo de execução.

Estático vs. Dinâmico

```
// C++ — tipagem estática:
int n = "ola";    // ERRO em tempo de compilação

# Python — tipagem dinâmica:
n = "ola"
n + 1              # TypeError em tempo de EXECUÇÃO
```

Eixo 2 — Tipagem Forte vs. Fraca

Tipagem forte significa que conversões implícitas entre tipos incompatíveis são proibidas ou altamente restritas. C++ é predominantemente forte (narrowing com {} é erro), mas

herda algumas coerções de C (int para float implicitamente). Python é forte dinâmico: "1" + 1 é erro. JavaScript é fraco dinâmico: "1" + 1 é "11" (coerção silenciosa).

Eixo 3 — Single Dispatch vs. Multiple Dispatch

Em C++, Java e Python, o método chamado depende do tipo do objeto receptor (único argumento virtual) — isso é single dispatch. Em linguagens com multiple dispatch (Common Lisp, Julia, Clojure), o método chamado depende dos tipos de todos os argumentos. Em C++, multiple dispatch pode ser simulado com visitor pattern ou std::visit sobre std::variant.

3.3 Mecanismos de Reuso: Herança, Composição e Traits

Mecanismo	Como funciona	Exemplos	C++ equivalente
Herança de classes	Subclasse herda implementação e interface da superclasse	C++, Java, Python	class B : public A
Composição	Objeto contém referência a outro; delega comportamento	Todas as linguagens	Membro do tipo A em B
Interfaces / Protocolos	Contrato de métodos sem implementação	Java, C#, Swift, Go	Classe abstrata pura em C++
Traits / Mixins	Pacotes de comportamento reutilizáveis sem hierarquia fixa	Rust (traits), Scala, Ruby (modules)	CRTP, herança múltipla de interfaces

C++ suporta todos esses mecanismos de alguma forma. A escolha entre herança e composição é uma das decisões de projeto mais importantes — e será explorada em profundidade na Unidade II.

3.4 Posicionando as Linguagens

Linguagem	Tipagem	Herança	Despacho dinâmico	Genérico
C++	Estática forte	Simples e múltipla	virtual (vtable)	Templates (Turing-completos)
Java	Estática forte	Simples (classes) + múltipla (interfaces)	Toda chamada de método	Generics (type erasure)
Python	Dinâmica forte	Múltipla (MRO)	Toda chamada (duck typing)	Implícito (duck typing)
JavaScript	Dinâmica fraca	Prototípica	Toda chamada	Implícito

Linguagem	Tipagem	Herança	Despacho dinâmico	Genérico
Rust	Estática forte	Sem herança de dados	Traits (vtable ou static)	Generics + traits (monomorphization)
Go	Estática forte	Sem herança	Interfaces implícitas	Generics (Go 1.18+)

**LLM:**

Prompt útil para este capítulo:

"Compare as implementações do padrão Observer em C++ com virtual functions, em Python com duck typing e em Rust com traits. Mostre código concreto para cada uma e explique os trade-offs de desempenho e segurança de tipos."

Analise criticamente a resposta: o LLM provavelmente acertará a estrutura geral, mas pode cometer erros sutis no código Rust (lifetimes) ou no código C++ (não usar smart pointers). Revise com atenção.

Exercícios Propostos

1. Pesquise o conceito de "duck typing" em Python. Escreva um exemplo em Python e um equivalente em C++ usando templates que demonstrem a mesma ideia. Explique as diferenças de quando os erros são detectados.
2. Java não permite herança múltipla de classes, mas permite implementar múltiplas interfaces. Qual problema isso resolve em relação ao problema do diamante? Como C++ lida com o mesmo problema?
3. Rust não tem herança de dados, mas tem traits. Pesquise como o padrão Iterator é implementado em Rust usando traits e compare com o uso de classes abstratas em C++.
4. Explique por que JavaScript é considerado de tipagem "fraca": dê três exemplos de coerção implícita silenciosa que podem causar bugs. Como TypeScript resolve esse problema?

Capítulo 4 — Infraestrutura do Programador C++

🎯 Objetivos deste capítulo

- ▶ Compilar programas C++ com GCC/Clang usando os flags corretos para a disciplina.
- ▶ Criar projetos organizados com CMake.
- ▶ Depurar programas com GDB e interpretar mensagens de erro comuns.
- ▶ Usar AddressSanitizer e UndefinedBehaviorSanitizer para detectar erros em tempo de execução.

4.1 O Modelo de Compilação de C++

Ao contrário de Python (interpretado) e Java (compilado para bytecode JVM), C++ é compilado diretamente para código de máquina (Figura 3).



Figura 3. Uma visualização do pipeline de desenvolvimento de C++, onde os streams de código (arquivos de origem) são forjados em blocos hexagonais (arquivos de objeto) por compiladores, e o "Linker" os une em um núcleo neon executável.

O processo tem três etapas: pré-processamento (expansão de `#include` e `#define`), compilação (C++ → objeto .o) e ligação (linking — combina objetos em executável).

Comandos de compilação essenciais

```
# Compilar e ligar em um só passo (simples, para arquivos únicos):
g++ -std=c++17 -Wall -Wextra -Wpedantic -o programa main.cpp

# Flags recomendados para desenvolvimento nesta disciplina:
g++ -std=c++17 -Wall -Wextra -Wpedantic \
    -fsanitize=address,undefined \
    -g -O0 \
    -o programa main.cpp

# Flags para release (otimizado):
g++ -std=c++17 -O2 -DNDEBUG -o programa main.cpp
```

Flag	Significado
-std=c++17	Usa o padrão C++17 (obrigatório na disciplina)

Flag	Significado
-Wall	Ativa a maioria dos warnings úteis
-Wextra	Ativa warnings adicionais além de -Wall
-Wpedantic	Exige conformidade estrita com o padrão
-g	Inclui informações de depuração (necessário para GDB)
-O0	Sem otimização — melhor para depuração
-O2	Otimização padrão para release
-fsanitize=address	Ativa AddressSanitizer (detecta vazamentos e acessos inválidos)
-fsanitize=undefined	Ativa UndefinedBehaviorSanitizer (detecta UB em runtime)

⚠ Cuidado:

NUNCA entregue código com warnings não resolvidos. -Wall -Wextra -Wpedantic são seus melhores aliados. Um warning silenciado é uma bug esperando acontecer.

4.2 CMake

CMake não compila código diretamente — ele gera arquivos de build (Makefiles, projetos Ninja, Visual Studio) a partir de uma descrição em CMakeLists.txt. É o padrão industrial para projetos C++ e a infraestrutura usada em todos os exemplos desta disciplina.

Estrutura mínima de projeto

Estrutura de diretórios recomendada

```
meu_projeto/
├── CMakeLists.txt
├── src/
│   ├── main.cpp
│   └── student.cpp
└── include/
    └── student.hpp
```

CMakeLists.txt mínimo

```
# CMakeLists.txt – versão mínima para a disciplina
cmake_minimum_required(VERSION 3.20)
project(meu_projeto CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)    # desativa extensões GNU não-padrão

# Flags de compilação
add_compile_options(
    -Wall -Wextra -Wpedantic
    ${<CONFIG:Debug>:-fsanitize=address,undefined}
    ${<CONFIG:Debug>:-g}
)
```

```
add_executable(meu_projeto src/main.cpp src/student.cpp)
target_include_directories(meu_projeto PRIVATE include)
```

Comandos CMake no terminal

```
# Como usar:
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Debug    # configura
cmake --build .                        # compila
./meu_projeto                          # executa
```

🎵 Sintonia:

O projeto Sintonia (v0.1 em diante) usa exatamente essa estrutura de CMake. Cada versão do Sintonia tem seu próprio CMakeLists.txt que herda configurações do CMakeLists.txt raiz via add_subdirectory().

4.3 GDB — Depuração Básica

GDB (GNU Debugger) permite executar o programa passo a passo, inspecionar variáveis e encontrar onde e por que o programa trava ou produz resultados errados. Compile sempre com -g para usar o GDB.

Comando GDB	Ação
<code>gdb ./programa</code>	Inicia o GDB com o programa
<code>run</code>	Executa o programa
<code>break main</code>	Define breakpoint na função main
<code>break student.cpp:42</code>	Define breakpoint na linha 42 do arquivo
<code>next (n)</code>	Avança uma linha (sem entrar em funções)
<code>step (s)</code>	Avança uma linha (entra em funções)
<code>print variavel</code>	Exibe o valor de uma variável
<code>backtrace (bt)</code>	Mostra a pilha de chamadas (onde estamos)
<code>quit (q)</code>	Sai do GDB

4.4 Sanitizers — Detecção de Erros em Runtime

Os sanitizers são instrumentações adicionadas pelo compilador que detectam classes de bugs em tempo de execução. São inestimáveis durante o desenvolvimento.

AddressSanitizer (ASan)

Detecta: uso de memória após liberação (use-after-free), leitura/escrita fora dos limites de arrays, vazamentos de memória, uso de variáveis não inicializadas na stack.

ASan detecta acesso fora dos limites

```
// Bug clássico detectado pelo ASan:
int* p = new int[5];
p[10] = 42; // escrita fora do limite – ASan detecta e aborta com relatório
```

```
delete[] p;
```

UndefinedBehaviorSanitizer (UBSan)

Detecta: overflow de inteiros com sinal, desreferência de ponteiro nulo, deslocamentos bit a bit inválidos, conversões de ponto flutuante inválidas.

UBSan detecta overflow de inteiro

```
// UB clássico detectado pelo UBSan:
int a = INT_MAX;
int b = a + 1;    // overflow de inteiro com sinal – UB! UBSan detecta
```

Boa prática:

Compile e execute seus projetos com `-fsanitize=address,undefined` durante todo o desenvolvimento. O custo de desempenho (~2x mais lento) é irrelevante em ambiente de aprendizado; o benefício de detectar bugs é imenso.

4.5 Transição de Python/C para C++

Conceito	Python	C	C++
Variável	<code>x = 42</code> (dinâmico)	<code>int x = 42;</code>	<code>int x = 42;</code> ou <code>auto x = 42;</code>
String	<code>str</code> — imutável	<code>char[]</code> — manual	<code>std::string</code> — automático
Lista/Array	<code>list</code>	array estático	<code>std::vector<T></code>
Dicionário	<code>dict</code>	— (implementação manual)	<code>std::map<K,V></code> ou <code>std::unordered_map<K,V></code>
Função	<code>def f(x):</code>	tipo <code>f(tipo x)</code>	tipo <code>f(tipo x)</code> ou <code>auto f(auto x)</code>
Classe	<code>class C:</code>	<code>struct</code> (sem métodos)	<code>class C { ... };</code>
Importar	<code>import módulo</code>	<code>#include <header.h></code>	<code>#include <header.hpp></code>
Executar	<code>python prog.py</code>	<code>gcc ... && ./prog</code>	<code>g++ ... && ./prog</code> (ou <code>cmake</code>)

Exercícios Propostos

1. Crie um projeto CMake mínimo com um arquivo `main.cpp` que imprime "Sintonia v0.0 ativo". Configure os flags `-Wall -Wextra` e `-fsanitize=address,undefined` no modo Debug. Compile e execute.
2. Compile o programa a seguir com ASan ativado e interprete o relatório gerado: `int* p = new int[3]; p[5] = 0; delete[] p;`. O que o relatório informa sobre o tipo de erro, o arquivo e a linha?

- 3.** Use GDB para depurar um programa que calcula a soma de elementos de um array mas produz resultado errado. Configure um breakpoint no loop, inspecione as variáveis a cada iteração e encontre o bug.
- 4.** Escreva um CMakeLists.txt para um projeto com dois arquivos de origem (main.cpp e calculator.cpp), um diretório de includes, flags de sanitizers no modo Debug e sem sanitizers no modo Release.

Capítulo 5 — Git e GitHub como Ambiente de Trabalho

Objetivos deste capítulo

- ▶ Usar Git para versionar código individualmente: init, add, commit, log, diff.
- ▶ Trabalhar com branches para isolar funcionalidades e experimentos.
- ▶ Colaborar via GitHub: fork, clone, pull request, code review.
- ▶ Interpretar e resolver conflitos de merge.
- ▶ Compreender o fluxo de trabalho da disciplina (repositório por atividade, PR para entrega).

5.1 Por que Versionar Código?

Desenvolvimento de software sem controle de versão é como escrever um documento importante sem nunca salvar um rascunho. Com Git: você pode desfazer qualquer mudança, comparar versões, trabalhar em paralelo em funcionalidades diferentes sem conflito, e colaborar com outros sem sobrescrever o trabalho alheio.

No contexto desta disciplina, Git e GitHub são a infraestrutura pedagógica: exercícios são entregues como commits em repositórios, pull requests são revisados como parte do processo formativo.

5.2 Conceitos Fundamentais

Conceito	O que é
Repositório (repo)	Diretório com todo o histórico do projeto (.git/)
Commit	Snapshot do estado do projeto em um dado momento, com mensagem descritiva
Branch	Linha de desenvolvimento independente; permite trabalhar em paralelo
HEAD	Ponteiro para o commit atual (geralmente o topo da branch corrente)
Remote	Repositório remoto (no GitHub); origin é o nome convencional
Clone	Cópia local de um repositório remoto
Pull Request (PR)	Pedido para integrar uma branch em outra; ponto de code review
Merge	Integração de duas branches
Conflict	Quando duas branches modificaram a mesma linha — requer resolução manual

5.3 Fluxo de Trabalho Individual

Comandos Git essenciais — fluxo individual

```
# 1. Criar repositório local
git init meu_projeto
```

```
cd meu_projeto

# 2. Configurar identidade (uma vez por máquina)
git config --global user.name "Seu Nome"
git config --global user.email "voce@email.com"

# 3. Adicionar arquivos à staging area
git add main.cpp
git add .          # adiciona todos os arquivos rastreáveis

# 4. Criar commit com mensagem descritiva
git commit -m "feat: adiciona classe student com encapsulamento"

# 5. Ver histórico
git log --oneline

# 6. Ver diferenças antes de commitar
git diff
git diff --staged  # diferenças já na staging area
```

Boa prática:

Escreva mensagens de commit descritivas no imperativo: "adiciona validação de nota" em vez de "validação adicionada" ou "mudanças". Prefixos úteis: feat: (nova funcionalidade), fix: (correção), docs: (documentação), test: (testes), refactor: (refatoração sem mudança de comportamento).

5.4 Branches e Merge

Branches — criar, commitar, mergear

```
# Criar e mudar para nova branch
git checkout -b feature/smart-pointers
# ou (forma moderna):
git switch -c feature/smart-pointers

# Ver todas as branches
git branch

# Commitar na nova branch
git add . && git commit -m "feat: implementa AudioBuffer com unique_ptr"

# Voltar para a branch principal
git switch main

# Integrar (merge) a feature
git merge feature/smart-pointers

# Deletar branch após merge
git branch -d feature/smart-pointers
```

5.5 GitHub — Colaboração e Entrega

GitHub — push, pull, fork e pull request

```
# Clonar repositório do GitHub
git clone https://github.com/usuario/poo-ufpb.git
```



```
# Enviar commits para o GitHub
git push origin main
git push origin feature/smart-pointers

# Baixar atualizações do remoto
git pull origin main

# Fluxo para entregar atividade como PR:
# 1. Fork do repositório da disciplina no GitHub (interface web)
# 2. Clone do fork: git clone https://github.com/seu-usuario/repo.git
# 3. Criar branch: git switch -c atividade-1-student
# 4. Desenvolver, commitar, push
# 5. Abrir Pull Request no GitHub (interface web)
```

5.6 .gitignore para Projetos C++

.gitignore — arquivo essencial para projetos C++/CMake

```
# .gitignore padrão para projetos C++ com CMake

# Diretório de build
build/
cmake-build-*/

# Executáveis e objetos compilados
*.o
*.a
*.so
*.out

# IDE
.vscode/
.idea/
*.user

# macOS
.DS_Store
```

Exercícios Propostos

1. Crie um repositório Git local para o projeto Sintonia v0.1. Faça pelo menos três commits com mensagens descritivas representando etapas de desenvolvimento (ex.: "feat: adiciona struct sample", "feat: implementa class track", "test: adiciona assert de tamanho de track").
2. Simule um conflito de merge: crie duas branches que modificam a mesma linha de um arquivo, tente fazer merge e resolva o conflito manualmente. Descreva o processo com prints dos comandos usados.
3. Publique seu repositório Sintonia no GitHub. Configure um arquivo .gitignore adequado para C++/CMake e verifique no GitHub que nenhum arquivo de build foi enviado.

4. Pesquise o conceito de "Git rebase" e explique a diferença entre rebase e merge. Em que situação cada um é mais adequado? Por que rebase pode ser perigoso em branches compartilhadas?

Capítulo 6 — Assistentes de Programação Baseados em LLM como Copiloto

🎯 Objetivos deste capítulo

- ▶ Compreender o que são LLMs e como funcionam como assistentes de programação.
- ▶ Escrever prompts eficazes para tarefas de modelagem OO, implementação e refatoração.
- ▶ Avaliar criticamente código gerado por LLMs sob a ótica de correção e boas práticas.
- ▶ Reconhecer os limites, riscos e responsabilidades no uso de assistentes de IA.

6.1 O que São LLMs e Como Funcionam

Modelos de Linguagem de Grande Escala (Large Language Models — LLMs) são sistemas de IA treinados em imensos corpus de texto (incluindo código-fonte de repositórios públicos). Eles aprendem padrões estatísticos de linguagem e os aplicam para completar ou gerar texto — incluindo código — que seja coerente com o contexto fornecido.

É crucial entender o que os LLMs não são: eles não "entendem" código no sentido semântico. Eles não executam o código que geram — não sabem se ele compila ou produz o resultado correto. Eles são muito bons em produzir texto que parece correto; verificar se é correto é sua responsabilidade.

LLMs são bons em...	LLMs (ainda) são ruins em...
Gerar esqueleto de classes a partir de especificação em linguagem natural	Raciocinar sobre complexidade algorítmica exata
Explicar conceitos e padrões de código	Garantir ausência de bugs em lógica complexa
Sugerir nomes de métodos e estrutura de interfaces	Manter consistência com código que não está no contexto
Gerar testes unitários para casos típicos	Descobrir edge cases não óbvios
Refatorar código seguindo padrões conhecidos	Raciocinar sobre efeitos de concorrência ou escala
Explicar mensagens de erro do compilador	Saber se uma biblioteca específica está disponível

6.2 Prompts Eficazes para POO

A qualidade da resposta de um LLM depende fortemente da qualidade do prompt. Prompts vagos geram respostas genéricas; prompts específicos e contextualizados geram respostas úteis.

Estrutura de um bom prompt de programação

- Contexto: qual é o projeto, linguagem, padrão (C++17, snake_case, identificadores em inglês, comentários em português).

- Restrições: o que não pode ser usado (ex.: sem new/delete crus, use smart pointers).
- Tarefa específica: o que exatamente deve ser gerado.
- Formato da saída: código com comentários, com testes, com diagrama UML.

Exemplo: prompt fraco vs. prompt forte

```
// PROMPT FRACO:
"Escreva uma classe para gerenciar um buffer de áudio em C++"

// PROMPT FORTE:
"Preciso de uma classe audio_buffer em C++17 para o projeto Sintonia.
Convenções: snake_case, identificadores em inglês, comentários em português.
Requisitos:
- Armazena samples como std::vector<float>
- Construtor parametrizado (size_t num_samples, float sample_rate)
- Construtor de cópia e atribuição de cópia (regra dos 5)
- Construtor de movimento e atribuição de movimento
- Destrutor (regra dos 5)
- Métodos: size(), sample_rate(), operator[], clear()
- Todos os métodos de acesso devem ter versão const
- Sem uso de new/delete crus – use std::vector
Gere a declaração em audio_buffer.hpp e a implementação em audio_buffer.cpp.
Adicione comentários explicativos em português em cada método."
```

6.3 Prompts por Contexto de Desenvolvimento OO

Modelagem — Esboço de Classes

Prompt de modelagem

"Dado o seguinte domínio de problema: [descrever domínio]
 Identifique as classes principais, seus atributos e métodos.
 Aplique os princípios de encapsulamento e responsabilidade única.
 Gere um diagrama UML de classes em sintaxe Mermaid e
 o esboço das declarações C++ correspondentes."

Refatoração — Aplicando SOLID

Prompt de refatoração com SOLID

"Analise o código abaixo e identifique violações dos princípios SOLID.
 Para cada violação: (1) nomeie o princípio violado, (2) explique a violação,
 (3) mostre o código refatorado que a corrige.
 Use C++17, snake_case, sem new/delete crus.
 [colar o código a ser analisado]"

Geração de Testes

Prompt de geração de testes Catch2

"Gere testes Catch2 para a classe audio_buffer abaixo.
 Cubra: construção com parâmetros válidos e inválidos,
 operações de cópia e movimento, acesso por índice válido e inválido,
 estado após clear(). Inclua pelo menos 8 casos de teste.",
 [colar declaração da classe]"

6.4 Análise Crítica de Código Gerado por LLMs

A seguir, um exemplo de código gerado por LLM que contém problemas reais.
 Identifique os problemas antes de ler a análise.

 **Código com problemas — para análise crítica**

```
// 🤖 CÓDIGO GERADO POR LLM — identifique os problemas:

class AudioBuffer {
    float* data;
    int size;
public:
    AudioBuffer(int s) : size(s) {
        data = new float[size];
    }
    ~AudioBuffer() { delete data; }    // ← problema 1

    float get(int i) { return data[i]; } // ← problema 2

    void copy_from(AudioBuffer& other) { // ← problema 3
        for (int i = 0; i < size; i++)
            data[i] = other.data[i];
    }
};
```

Problemas identificados:

- Problema 1: delete deveria ser delete[] para arrays alocados com new[].
- Problema 2: get() não valida i — acesso fora dos limites é UB. Não é const.
- Problema 3: copy_from() não verifica se os tamanhos são iguais — UB se other.size != size. Viola o princípio de que objetos devem ter interface segura.
- Problema geral: usa new/delete cru em vez de std::vector. Não segue a regra 0/3/5 — falta construtor de cópia e operador de atribuição, levando a double-free se copiado.
- Convenção: PascalCase (AudioBuffer) e identificadores em inglês (OK), mas comentários ausentes.

6.5 Limites, Riscos e Responsabilidades

- Alucinações: LLMs podem inventar APIs que não existem, como métodos de biblioteca que nunca foram implementados. Sempre verifique na documentação oficial.
- Licenciamento: código gerado pode ser derivado de código com licenças restritivas do conjunto de treinamento. Verifique a política de uso do seu LLM antes de usar em produção.
- Confidencialidade: nunca cole código proprietário, dados sensíveis ou credenciais em prompts para serviços externos.
- Responsabilidade: você é responsável por todo código que entrega, independentemente de ter sido gerado por IA. "O LLM gerou" não é justificativa para bug ou falha de segurança.
- Dependência: usar LLMs para tudo sem entender o que está sendo gerado cria uma dependência que limita seu crescimento como programador.

LLM:

Boa prática ao usar LLMs nesta disciplina:

1. Primeiro tente resolver o problema por conta própria.
2. Se usar LLM, analise cada linha do código gerado e entenda o que faz.
3. Compile e execute — nunca assuma que o código gerado está correto.
4. Execute com sanitizers ativados.
5. Escreva pelo menos um teste que valide o comportamento esperado.
6. Identifique e corrija qualquer problema encontrado.

Exercícios Propostos

1. Escreva um prompt para gerar a classe `track` do projeto Sintonia que contenha: lista de samples, taxa de amostragem, métodos de acesso, construtor e destrutor corretos sem `new/delete`. Execute o prompt em um LLM, analise o código gerado e liste os problemas encontrados.
2. O código a seguir foi gerado por um LLM. Identifique pelo menos 4 problemas:

```
class Stack { int* arr; int top; public: Stack() { arr = new int[100]; top=0; } void push(int x) { arr[top++]=x; } int pop() { return arr[--top]; } };
```
3. Crie um prompt que instrua um LLM a refatorar o código problemático do exercício anterior aplicando: `std::vector` em vez de `new[]`, validação nos métodos, `const-correctness` e identifique se o LLM cometeu erros na refatoração.
4. Pesquise as políticas de uso de código gerado por IA nas principais licenças de software livre (MIT, Apache 2.0, GPL). Em quais casos o uso de código gerado por LLM pode ser problemático do ponto de vista legal?

Capítulo 7 — UML Leve como Ferramenta de Projeto

Objetivos deste capítulo

- ▶ Ler e produzir diagramas de classes UML básicos.
- ▶ Representar herança, composição, agregação e dependência em UML.
- ▶ Ler e produzir diagramas de sequência UML simples.
- ▶ Usar UML como rascunho de projeto antes de escrever código.

7.1 Por que UML? A Modelagem Antes do Código

UML (Unified Modeling Language) é uma notação gráfica padronizada para modelar sistemas de software. Nesta disciplina usamos UML "leve" — apenas os elementos essenciais dos diagramas de classes e de sequência, sem burocracia.

O valor de fazer um diagrama de classes antes de codificar é semelhante ao de fazer um esboço arquitetônico antes de construir: é muito mais barato corrigir um retângulo no papel do que refatorar 500 linhas de código. Ferramentas como Mermaid (usada no site da disciplina), draw.io e PlantUML permitem criar diagramas rapidamente.

7.2 Diagrama de Classes — Notação Essencial

Uma classe é representada como um retângulo com três compartimentos: nome (topo), atributos (meio) e operações/métodos (fundo). A visibilidade é indicada por prefixos: + (public), - (private), # (protected).

Classe student em Mermaid

```
// Notação Mermaid — gera diagrama de classes automaticamente
classDiagram
    class student {
        -int id_
        -string name_
        -float grade1_
        -float grade2_
        -float grade3_
        +student(int id, string name)
        +set_grades(float g1, float g2, float g3) void
        +avg() float
        +print() void
    }
```



DIAGRAMA UML



Este diagrama ilustra Herança (elo), Composição Forte (diamante preenchido), Composição Fraca (diamante vazio), Dependência (seta tracejada) e Associação (linha simples).

7.3 Relacionamentos entre Classes

Relacionamento	Símbolo UML	Significado	Exemplo C++
Herança (generalização)	Seta fechada vazia —>	"é um" — subclasse herda da superclasse	<code>class grad_student : public student</code>
Composição forte	Losango preenchido ◆—	"tem um" — parte não existe sem o todo	<code>class car { engine engine_;};</code>
Composição fraca (agregação)	Losango vazio ◇—	"tem um" — parte pode existir sem o todo	<code>class team { vector<player*> members_;};</code>
Dependência (uso)	Seta tracejada -->	"usa" — métrica de acoplamento	Parâmetro ou variável local de outro tipo
Associação	Linha —	Relação estrutural genérica	Membro ponteiro sem propriedade

Diagrama de classes do Sintonia v0.3 em Mermaid

```
classDiagram
    class project {
        -vector~track~ tracks_
        +add_track(track t) void
        +mix() audio_buffer
    }
    class track {
        -audio_buffer buffer_
        -float sample_rate_
        +size() size_t
    }
```



```
class audio_buffer {
    -vector<float> samples_
    -float sample_rate_
    +operator[](size_t i) float
}
project "1" *-- "0..*" track : contém
track "1" *-- "1" audio_buffer : possui
```

🎵 Sintonia:

O diagrama acima representa a estrutura do Sintonia v0.3.

project é composto de tracks (composição forte — track não existe sem project).

track possui um audio_buffer (composição forte).

Esse diagrama deve ser desenhado ANTES de escrever o código de cada versão.

7.4 Diagrama de Sequência — Interação entre Objetos

O diagrama de sequência mostra como objetos interagem ao longo do tempo para realizar um caso de uso. É lido de cima para baixo, com o tempo fluindo para baixo e as mensagens trocadas horizontalmente entre as "lifelines" (linhas de vida).

Diagrama de sequência: criação e mix do Sintonia

```
sequenceDiagram
    participant main as main()
    participant p as project
    participant t as track
    participant b as audio_buffer

    main->>p: add_track(track_data)
    p->>t: construtor(buffer)
    t->>b: construtor(num_samples, sample_rate)
    b-->>t: buffer criado
    t-->>p: track criada
    p-->>main: ok
    main->>p: mix()
    loop para cada track
        p->>t: get_buffer()
        t->>b: operator[](i)
        b-->>t: sample[i]
        t-->>p: sample
    end
    p-->>main: audio_buffer final
```

7.5 Fluxo de Trabalho com UML Leve

1. Leia o enunciado do problema e sublinhe substantivos (candidatos a classes) e verbos (candidatos a métodos).
2. Esboce um diagrama de classes no papel ou em Mermaid: identifique atributos, métodos e relacionamentos.
3. Verifique os relacionamentos: é realmente "é um" (herança) ou "tem um" (composição)?
4. Para fluxos complexos, esboce um diagrama de sequência para o caso de uso principal.

5. Só então comece a escrever código — o diagrama é o contrato.
6. Atualize o diagrama quando a implementação revelar decisões de projeto não previstas.

**Boa prática:**

UML não é burocracia — é comunicação. Um diagrama de classes de 5 minutos no papel pode evitar 2 horas de refatoração. Mas não exagere: UML "leve" significa o suficiente para clarificar o design, não um manual técnico completo.

Exercícios Propostos

1. Desenhe em Mermaid o diagrama de classes para um sistema de biblioteca: Book (título, ISBN, ano), Author (nome, bio), Library (coleção de livros), Loan (livro, usuário, data). Indique os relacionamentos corretos.
2. Dado o diagrama de classes do Sintonia v0.3 mostrado neste capítulo, implemente as três classes em C++17 com os atributos e métodos indicados. Use `std::vector` para as coleções.
3. Escreva um diagrama de sequência em Mermaid para o processo de empréstimo de livro na biblioteca do exercício 1: usuário solicita empréstimo → biblioteca verifica disponibilidade → registra empréstimo → retorna confirmação.
4. Analise o seguinte cenário e identifique se a relação é herança ou composição: (a) Avião e Motor; (b) Animal e Mamífero; (c) Conta Bancária e Transação; (d) Forma Geométrica e Círculo; (e) Carro e Proprietário.

Capítulo 8 — Classes e Objetos em C++

🎯 Objetivos deste capítulo

- ▶ Declarar e definir classes em C++ com encapsulamento correto.
- ▶ Usar modificadores de acesso (public, protected, private) apropriadamente.
- ▶ Compreender o papel do ponteiro this.
- ▶ Aplicar const-correctness: métodos const, parâmetros const, referências const.
- ▶ Usar membros estáticos (static) para dados e comportamento compartilhados por todas as instâncias.

8.1 Declaração e Definição de Classes

Em C++, a convenção é separar a declaração (interface) em um arquivo de cabeçalho (.hpp) e a implementação (definição dos métodos) em um arquivo de implementação (.cpp). Isso reduz o tempo de recompilação e separa a interface do cliente da implementação.

sample.hpp — separação de interface e implementação

```
//sample.hpp — declaração (interface)
#pragma once
#include <cstdint>    // size_t

class sample {
public:
    sample();           // construtor padrão
    explicit sample(float amplitude,
                    float sample_rate); // construtor parametrizado

    float amplitude() const;           // getter (const — não modifica)
    float sample_rate() const;
    void set_amplitude(float a);       // setter (pode modificar)

    bool is_silent() const;

private:
    float amplitude_;
    float sample_rate_;

    static constexpr float MAX_AMP = 1.0f; // constante da classe
};
```

sample.cpp — implementação com validação

```
// sample.cpp — implementação
#include "sample.hpp"
#include <stdexcept>

/* Construtor padrão: amplitude zero, taxa padrão */
sample::sample() : amplitude_{0.0f}, sample_rate_{44100.0f} {}

/* Construtor parametrizado com validação */
sample::sample(float amplitude, float sample_rate)
    : amplitude_{0.0f}, sample_rate_{sample_rate}
{
    set_amplitude(amplitude); // reutiliza a validação do setter
```

```

}

/* Getter — não modifica o objeto (const) */
float sample::amplitude() const { return amplitude_; }
float sample::sample_rate() const { return sample_rate_; }

/* Setter com validação */
void sample::set_amplitude(float a) {
    if (a < -MAX_AMP || a > MAX_AMP)
        throw std::domain_error("amplitude fora de [-1, 1]");
    amplitude_ = a;
}

bool sample::is_silent() const { return amplitude_ == 0.0f; }

```

🎵 Sintonia:

sample é a primeira classe do projeto Sintonia (v0.1).

Observe: construtor parametrizado é explicit — evita conversões implícitas acidentais de float para sample.

O setter reutiliza a validação em vez de duplicá-la no construtor.

8.2 Modificadores de Acesso

Modificador	Quem pode acessar	Uso típico
public	Qualquer código	Interface da classe: construtores, getters, setters, operações
private	Apenas a própria classe	Dados internos, métodos auxiliares de implementação
protected	A própria classe e classes derivadas	Dados que subclasses precisam acessar diretamente

💡 Boa prática:

Regra geral: comece tudo como private. Torne public apenas o que é necessário para o contrato da classe. Use protected somente quando projetar herança — e mesmo assim, considere getters/setters protected em vez de dados diretamente protected.

8.3 O Ponteiro this

Dentro de qualquer método não-estático, this é um ponteiro implícito para o objeto no qual o método foi chamado. Raramente é necessário explicitá-lo — mas é útil para resolver ambiguidade de nomes e para retornar *this em interfaces fluentes.

this e interface fluente

```

class track {
public:
    track& set_name(const std::string& name) {

```

```

        this->name_ = name;    // this-> resolve ambiguidade se parâmetro se
chamar name_
        return *this;          // interface fluente: permite
track.set_name("x").set_gain(0.5f)
    }
    track& set_gain(float gain) {
        gain_ = gain;
        return *this;
    }
private:
    std::string name_;
    float gain_ = 1.0f;
};

// Uso:
track t;
t.set_name("drums").set_gain(0.8f); // encadeamento de chamadas

```

8.4 Const-Correctness

const-correctness significa usar `const` em todos os lugares onde um objeto não deve ser modificado. É uma das práticas mais importantes de C++ moderno — ela torna as intenções explícitas, permite otimizações do compilador e detecta bugs.

As formas de `const` em C++

```

// Método const: garante que NÃO modifica o objeto
float sample::amplitude() const { return amplitude_; }

// Parâmetro const&: aceita qualquer objeto (temporário ou não) sem copiar
void print_sample(const sample& s) {
    std::cout << s.amplitude() << "\n"; // só pode chamar métodos const
}

// Variável const: não pode ser reatribuída
const float MAX_RATE = 192000.0f;

// Ponteiro para const: não modifica o dado apontado
const float* ptr = &amplitude_;

// Ponteiro const: não pode apontar para outro endereço
float* const ptr2 = &amplitude_;

```

⚠ Cuidado:

Se um método logicamente não modifica o objeto, declare-o como `const`. Sem o `const`, você não pode chamar esse método em objetos `const` — o que limita a utilidade da classe em interfaces de leitura.

8.5 Membros Estáticos (`static`)

Membros `static` pertencem à classe, não a instâncias individuais. Uma variável `static` é compartilhada por todos os objetos da classe; um método `static` pode ser chamado sem criar um objeto.

Membros estáticos — contador de instâncias

```

class sample {

```

```
public:
    static int instance_count() { return count_; }
    static constexpr float DEFAULT_RATE = 44100.0f; // constante de classe

    sample() { ++count_; }
    ~sample() { --count_; }

private:
    static int count_; // compartilhado por TODAS as instâncias
    float amplitude_;
    float rate_;
};

// Definição fora da classe (obrigatório para static não-constexpr):
int sample::count_ = 0;

// Uso:
std::cout << sample::instance_count() << "\n"; // sem criar objeto
{
    sample s1, s2;
    std::cout << sample::instance_count() << "\n"; // 2
} // s1 e s2 destruídos
std::cout << sample::instance_count() << "\n"; // 0
```

Exercícios Propostos

1. Implemente a classe `track` para o Sintonia v0.1. Ela deve conter: `vector<sample> samples_`, `float sample_rate_`, `std::string name_`. Adicione métodos: construtor, `add_sample(sample s)`, `size() const`, `operator[](size_t i) const`, `name() const`. Separe em `.hpp` e `.cpp`.
2. Adicione const-correctness completa à classe `track`: revise todos os métodos e marque como `const` os que não modificam o objeto. Compile com `-Wall` e verifique que não há warnings.
3. Adicione um membro `static` a `sample` que conta o número de instâncias vivas. Escreva um teste no `main()` que cria 3 `samples`, imprime o contador, destrói um deles (usando escopo) e imprime o contador novamente.
4. Explique com um exemplo concreto por que o compilador recusa chamar um método não-const em um objeto const. O que acontece internamente? Como isso ajuda a detectar bugs?

Capítulo 9 — Ciclo de Vida de Objetos: Construtores, Destrutores e RAI

🎯 Objetivos deste capítulo

- ▶ Dominar os tipos de construtores: padrão, parametrizado, cópia, conversão.
- ▶ Compreender listas de inicialização e a ordem obrigatória de inicialização.
- ▶ Implementar destrutores corretamente e entender quando são chamados.
- ▶ Aplicar o idioma RAI para gerenciamento automático de recursos.

9.1 O que é o Ciclo de Vida de um Objeto?

Todo objeto em C++ passa por três fases (Figura 4): construção (alocação de memória + inicialização dos membros), uso (chamadas de métodos, acesso a atributos) e destruição (liberação de recursos + desalocação de memória).



Figura 4. Uma ilustração conceitual de como o C++ automatiza o gerenciamento de recursos, onde o scope de um objeto se fecha automaticamente ao sair de sua função, vedando e desativando o recurso interno (como a energia central).

O programador controla esse ciclo por meio de construtores e destrutores. A seguir discutiremos com detalhes esses mecanismos.

9.2 Tipos de Construtores

Construtor Padrão

Chamado quando o objeto é criado sem argumentos. Se não declarado e a classe não tiver outros construtores, o compilador gera um automaticamente (que faz nada). Membros de tipo primitivo ficam não-inicializados (valor indefinido).

Construtor padrão

```
class audio_buffer {
public:
    /* Construtor padrão: buffer vazio com taxa padrão */
    audio_buffer() : samples_[], sample_rate_{44100.0f} {}
    //      ^ lista de inicialização – preferível a atribuição no corpo
```

```
private:
    std::vector<float> samples_;
    float sample_rate_;
};

audio_buffer buf; // chama construtor padrão
```

Construtor Parametrizado

Construtor parametrizado com lista de inicialização

```
/* Construtor parametrizado: aloca buffer com num_samples zeros */
audio_buffer::audio_buffer(size_t num_samples, float sample_rate)
    : samples_(num_samples, 0.0f), // vetor de num_samples zeros
      sample_rate_{sample_rate}
{
    if (sample_rate <= 0.0f)
        throw std::invalid_argument("sample_rate deve ser positivo");
}

audio_buffer buf{44100, 48000.0f}; // 44100 amostras, 48 kHz
```

Construtor de Cópia

Chamado quando um objeto é inicializado a partir de outro do mesmo tipo: `AudioBuffer b2{b1}` ou `AudioBuffer b2 = b1`. Se não declarado, o compilador gera uma cópia membro a membro (funciona bem para classes sem recursos manuais).

Construtor de cópia

```
/* Construtor de cópia: cópia profunda (deep copy) */
audio_buffer::audio_buffer(const audio_buffer& other)
    : samples_{other.samples_}, // std::vector cuida da cópia
      sample_rate_{other.sample_rate_}
{}

audio_buffer b1{1024, 44100.0f};
audio_buffer b2{b1}; // cópia – b1 e b2 são independentes
```

9.3 Listas de Inicialização — Por que Usar Sempre

A lista de inicialização (`:` membro_{expr} antes do corpo do construtor) inicializa os membros antes que o corpo do construtor execute. Isso é mais eficiente (sem construção padrão seguida de atribuição) e é obrigatório para membros `const`, referências e classes sem construtor padrão.

Lista de inicialização vs. atribuição no corpo

```
class track {
public:
    // ❌ ERRADO: atribuição no corpo (construção padrão + atribuição)
    track(std::string name, float rate) {
        name_ = name; // copia; string já foi construída vazia
        rate_ = rate; // atribuição de float: OK mas inconsistente
    }

    // ✅ CORRETO: lista de inicialização (inicialização direta)
    track(std::string name, float rate)
        : name_{std::move(name)}, // move – sem cópia desnecessária
          rate_{rate}
    {}

private:
```



```
std::string name_;
float rate_;
};
```

⚠ Cuidado:

A ordem de inicialização na lista NÃO é a ordem que você escreve — é a ordem de declaração dos membros na classe. Escreva a lista na mesma ordem dos membros para evitar bugs sutis (e warnings do compilador com -Wall).

9.4 Destrutores e a Ordem de Destruição

O destrutor (~NomeDaClasse()) é chamado automaticamente quando o objeto sai de escopo (stack) ou quando delete é chamado (heap). A ordem de destruição é sempre o inverso da ordem de construção.

Ordem de construção e destruição

```
#include <iostream>

class resource {
public:
    resource(std::string name) : name_{std::move(name)} {
        std::cout << "Construindo: " << name_ << "\n";
    }
    ~resource() {
        std::cout << "Destruindo: " << name_ << "\n";
    }
private:
    std::string name_;
};

int main() {
    resource r1{"primeiro"};
    resource r2{"segundo"};
    {
        resource r3{"terceiro (escopo interno)"};
    } // r3 destruído aqui
    return 0;
} // r2 destruído, depois r1

// Saída:
// Construindo: primeiro
// Construindo: segundo
// Construindo: terceiro (escopo interno)
// Destruindo: terceiro (escopo interno)
// Destruindo: segundo
// Destruindo: primeiro
```

9.5 RAI — Resource Acquisition Is Initialization

RAI é o idioma mais importante de C++ moderno. A ideia: ao adquirir um recurso (arquivo, mutex, conexão de rede, memória dinâmica), inicialize-o no construtor; libere-o no destrutor. Como o destrutor é chamado automaticamente ao sair do escopo, o recurso é SEMPRE liberado — mesmo se uma exceção for lançada.

RAI com std::ifstream

```
// Sem RAI: liberação manual – sujeita a esquecimento e exceções
void read_file_bad(const std::string& path) {
    FILE* f = fopen(path.c_str(), "r");
    // ... processar ...
    // E se uma exceção for lançada aqui?
    fclose(f); // pode não executar!
}

// Com RAI: std::ifstream fecha sozinho
#include <fstream>
void read_file_good(const std::string& path) {
    std::ifstream file{path}; // abre no construtor
    if (!file) throw std::runtime_error("arquivo não encontrado");
    std::string line;
    while (std::getline(file, line))
        std::cout << line << "\n";
} // file.close() chamado AUTOMATICAMENTE no destrutor
```

RAI para leitura de arquivo WAV — Sintonia v0.3

```
// RAI para um arquivo WAV – usado no Sintonia v0.3
class wav_file {
public:
    explicit wav_file(const std::string& path)
        : stream_{path, std::ios::binary}
    {
        if (!stream_)
            throw std::runtime_error("nao foi possivel abrir: " + path);
        read_header();
    }
    // Destrutor implícito: stream_ fecha automaticamente

    size_t num_samples() const { return num_samples_; }
    float sample_rate() const { return sample_rate_; }

private:
    std::ifstream stream_;
    size_t num_samples_ = 0;
    float sample_rate_ = 0.0f;

    void read_header(); // lê cabeçalho RIFF WAV
};
```

Boa prática:

Use sempre tipos RAI da STL: `std::ifstream/ofstream` para arquivos, `std::unique_ptr` para memória dinâmica, `std::lock_guard` para mutexes. Você raramente precisa escrever seu próprio RAI.

Exercícios Propostos

1. Implemente a classe `audio_buffer` completa com: construtor padrão, construtor parametrizado (`size_t`, `float`), construtor de cópia, destrutor (mesmo que vazio, para imprimir uma mensagem de debug). Use lista de inicialização em todos os construtores. Compile com `-Wall` e corrija todos os warnings.

- 2.** Adicione ao `audio_buffer` um membro `static` que conta o número total de samples criados ao longo do programa (somando os tamanhos dos vetores de todos os buffers criados). Escreva um teste que cria e destrói buffers e verifica o contador.
- 3.** Escreva uma classe `file_guard` que usa `RAII` para garantir que um arquivo `FILE*` seja fechado quando o guarda sai de escopo. O construtor abre o arquivo (lança exceção se falhar), o destrutor chama `fclose()`. Demonstre que o arquivo é fechado mesmo se uma exceção for lançada no meio do processamento.
- 4.** Explique com código o que acontece quando um objeto com um `std::vector` como membro é copiado. O vetor é copiado raso (mesma memória) ou profundo (nova memória)? Como isso muda se o membro fosse um raw pointer `int*`?

UNIDADE II

Ciclo de Vida, Herança e Polimorfismo

RAll, smart pointers, movimento, herança, vtables e polimorfismo dinâmico

Capítulo 10 — RAII e a Regra dos Zero/Três/Cinco

🎯 Objetivos deste capítulo

- ▶ Compreender a regra do zero: quando o compilador gera tudo corretamente.
- ▶ Aplicar a regra dos três: destrutor, cópia e atribuição de cópia.
- ▶ Aplicar a regra dos cinco: acrescentar movimento ao conjunto.
- ▶ Distinguir semântica de valor de semântica de referência.

10.1 O que o Compilador Gera Automaticamente

Para qualquer classe, o compilador pode gerar automaticamente até seis operações especiais: construtor padrão, destrutor, construtor de cópia, operador de atribuição por cópia, construtor de movimento e operador de atribuição por movimento. Quando e o quê ele gera dependem do que você declarou.

Se você declarar...	O compilador NÃO gera...
Qualquer construtor	Construtor padrão
Destrutor	Construtor/atribuição de movimento
Construtor de cópia	Construtor/atribuição de movimento
Atribuição de cópia	Construtor/atribuição de movimento
Construtor de movimento	Construtor/atribuição de cópia
Atribuição de movimento	Construtor/atribuição de cópia

10.2 Regra do Zero

Se a sua classe gerencia apenas recursos via tipos RAII da STL (`std::vector`, `std::string`, `std::unique_ptr` etc.), declare zero das seis operações especiais. O compilador gera versões corretas que chamam as operações corretas dos membros.

Regra do Zero — classe sem operações especiais declaradas

```
// Regra do ZERO: nenhuma operação especial declarada
class track {
public:
    track(std::string name, float rate)
        : name_{std::move(name)}, rate_{rate} {}

    // Sem destrutor declarado: std::string e std::vector cuidam de tudo
    // Sem construtor de cópia: gerado pelo compilador (copia membros)
    // Sem operador de atribuição: gerado pelo compilador
    // Sem construtor de movimento: gerado pelo compilador
    // Sem operador de atribuição de movimento: gerado

    void add_sample(float s) { samples_.push_back(s); }
    size_t size() const { return samples_.size(); }

private:
    std::string      name_;
```

```
float          rate_;
std::vector<float> samples_;
};
```

Boa prática:

Prefira sempre a regra do zero. Se você precisa declarar qualquer uma das seis operações especiais, provavelmente está gerenciando um recurso que deveria ser encapsulado em sua própria classe RAI — e então essa classe interna seguiria a regra dos cinco.

10.3 Regra dos Três (Pré-C++11)

Se uma classe gerencia um recurso manualmente (raw pointer, file descriptor, socket), ela precisa de destrutor, construtor de cópia e operador de atribuição de cópia — os três, sempre juntos. Sem todos os três, operações de cópia padrão geram shallow copies que levam a double-free ou use-after-free.

Regra dos três — gestão manual de recurso

```
// Classe gerenciando raw pointer – regra dos TRÊS
class raw_buffer {
public:
    explicit raw_buffer(size_t size)
        : size_{size}, data_{new float[size]} {}

    /* 1. Destrutor: libera o recurso */
    ~raw_buffer() { delete[] data_; }

    /* 2. Construtor de cópia: cópia profunda */
    raw_buffer(const raw_buffer& other)
        : size_{other.size_}, data_{new float[other.size_]}
    {
        std::copy(other.data_, other.data_ + other.size_, data_);
    }

    /* 3. Operador de atribuição de cópia */
    raw_buffer& operator=(const raw_buffer& other) {
        if (this == &other) return *this; // autoatribuição
        float* new_data = new float[other.size_]; // aloca primeiro
        std::copy(other.data_, other.data_ + other.size_, new_data);
        delete[] data_; // libera o antigo
        data_ = new_data; // assume o novo
        size_ = other.size_;
        return *this;
    }

private:
    size_t size_;
    float* data_;
};
```

⚠ Cuidado:

Este exemplo usa raw pointer para fins didáticos — mostrar a regra dos três. Em código real, use `std::vector<float>` e a regra do zero. Se você se pega implementando a regra dos três, pergunte-se: existe um tipo RAII da STL que faz isso?

10.4 Regra dos Cinco (C++11 em diante)

Com C++11, o compilador pode gerar operações de movimento (construtor de movimento e atribuição de movimento). Se você declara qualquer uma das três operações da regra dos três, o compilador não gera as de movimento. Para ter todas as seis corretas, declare todas as cinco.

Regra dos cinco — todos os cinco especiais

```
class raw_buffer {
public:
    explicit raw_buffer(size_t size)
        : size_{size}, data_{new float[size]} {}

    /* 1. Destrutor */
    ~raw_buffer() { delete[] data_; }

    /* 2. Construtor de cópia */
    raw_buffer(const raw_buffer& other)
        : size_{other.size_}, data_{new float[other.size_]}
    { std::copy(other.data_, other.data_ + other.size_, data_); }

    /* 3. Operador de atribuição de cópia (idioma copy-and-swap) */
    raw_buffer& operator=(raw_buffer other) { // passa por valor: já copia
        std::swap(size_, other.size_);
        std::swap(data_, other.data_);
        return *this; // other (com o dado antigo) é destruído aqui
    }

    /* 4. Construtor de movimento: rouba os recursos, deixa other vazio */
    raw_buffer(raw_buffer&& other) noexcept
        : size_{other.size_}, data_{other.data_}
    {
        other.size_ = 0;
        other.data_ = nullptr; // other fica em estado válido mas vazio
    }

    /* 5. Operador de atribuição de movimento */
    raw_buffer& operator=(raw_buffer&& other) noexcept {
        if (this == &other) return *this;
        delete[] data_;
        data_ = other.data_;
        size_ = other.size_;
        other.data_ = nullptr;
        other.size_ = 0;
        return *this;
    }

private:
    size_t size_;
    float* data_;
};
```

10.5 Semântica de Valor vs. Referência

Tipos com semântica de valor se comportam como `int`: copiar cria um objeto independente, modificar a cópia não afeta o original. É o comportamento padrão de classes C++ com a regra do zero corretamente aplicada. Semântica de referência é quando múltiplos handles apontam para o mesmo dado subjacente — como `shared_ptr`.

Exercícios Propostos

1. Implemente a regra dos zero para a classe `audio_buffer` usando `std::vector<float>` internamente. Verifique que cópia, atribuição, movimento e destruição funcionam corretamente sem nenhum código extra.
2. Implemente a regra dos cinco completa para a classe `raw_buffer` mostrada no capítulo, usando o idioma `copy-and-swap` para o operador de atribuição de cópia. Escreva testes que verificam: (a) cópia produz objeto independente; (b) movimento esvazia o objeto de origem; (c) não há `double-free`.
3. Explique o que aconteceria se `raw_buffer` tivesse apenas o destrutor declarado (sem construtor de cópia nem operador de atribuição). Demonstre o bug com código.
4. Por que as operações de movimento devem ser marcadas `noexcept`? Pesquise o impacto de `noexcept` em `std::vector::push_back` — quando ele usa movimento vs. cópia.

Capítulo 11 — Smart Pointers

🎯 Objetivos deste capítulo

- ▶ Usar `std::unique_ptr` para propriedade exclusiva.
- ▶ Usar `std::shared_ptr` para propriedade compartilhada.
- ▶ Usar `std::weak_ptr` para quebrar ciclos de referência.
- ▶ Escolher o smart pointer correto para cada situação.

11.1 Por que Smart Pointers?

Raw pointers (`int*`, `float*`) não comunicam propriedade: quem é responsável por chamar `delete`? Smart pointers encapsulam um raw pointer e chamam `delete` automaticamente quando não são mais necessários — RAII aplicado à memória dinâmica.

Smart Pointer	Propriedade	Contagem de refs	Custo
<code>unique_ptr<T></code>	Exclusiva — um único dono	Não	Zero overhead vs. raw pointer
<code>shared_ptr<T></code>	Compartilhada — múltiplos donos	Sim (atômico)	Alocação extra + overhead atômico
<code>weak_ptr<T></code>	Nenhuma — observador	Não conta como strong ref	Similar ao <code>shared_ptr</code>

11.2 `unique_ptr` — Propriedade Exclusiva

`unique_ptr` — propriedade exclusiva e transferência

```
#include <memory>
#include <iostream>

class effect {
public:
    virtual void process(float& sample) = 0;
    virtual ~effect() = default;
};

class gain : public effect {
    float factor_;
public:
    explicit gain(float f) : factor_{f} {}
    void process(float& s) override { s *= factor_; }
};

int main() {
    // make_unique: forma correta de criar unique_ptr (nunca use new diretamente)
    auto eff = std::make_unique<gain>(0.5f);
    float sample = 1.0f;
    eff->process(sample);
    std::cout << sample << "\n"; // 0.5

    // Mover propriedade (transferência): eff fica vazio
    auto eff2 = std::move(eff);
```

```
// eff.get() == nullptr agora

return 0;
} // eff2 destruído aqui — gain liberado automaticamente
```

11.3 shared_ptr — Propriedade Compartilhada

shared_ptr — contagem de referências

```
#include <memory>
#include <vector>

// Recurso compartilhado: múltiplas tracks usando o mesmo preset de efeitos
auto preset = std::make_shared<gain>(0.8f);

std::vector<std::shared_ptr<effect>> chain;
chain.push_back(preset); // use_count = 2

{
    auto chain2 = chain; // cópia: use_count = 3
} // chain2 destruído: use_count = 2

chain.clear(); // use_count = 1
// preset sai de escopo: use_count = 0 -> gain liberado
```

⚠ Cuidado:

Nunca crie dois `shared_ptr` independentes a partir do mesmo raw pointer — isso cria dois contadores de referência independentes, resultando em double-free. Use sempre `make_shared` ou `shared_from_this`.

11.4 weak_ptr — Quebrando Ciclos

Se dois objetos se apontam mutuamente com `shared_ptr`, nenhum será destruído (`use_count` nunca chega a zero — ciclo de referência). `weak_ptr` observa um `shared_ptr` sem incrementar a contagem.

weak_ptr — observador sem posse

```
struct node {
    int value;
    std::shared_ptr<node> next; // forte: ciclo se bidirecional
    std::weak_ptr<node> prev; // fraca: não causa ciclo
};

// Para usar um weak_ptr, converta para shared_ptr temporariamente:
auto sp = weak.lock(); // retorna shared_ptr ou nullptr se objeto foi destruído
if (sp) {
    sp->value = 42; // seguro: objeto ainda existe
}
```

11.5 Regra de Escolha

- `unique_ptr`: padrão para posse exclusiva de objetos polimórficos em coleções de efeitos, árvores, grafos sem ciclos.

- `shared_ptr`: quando múltiplos donos precisam sobreviver independentemente — presets compartilhados, cache de recursos.
- `weak_ptr`: para referências para trás em árvores (pai observa filho sem possuir), observers, caches sem propriedade.
- Referência (`T&`): quando não há transferência de propriedade — parâmetros de funções, loops.
- Raw pointer (`T*`): apenas em interfaces de baixo nível, código legado ou quando o tempo de vida é garantido por outro mecanismo.

🎵 Sintonia:

No Sintonia v1.1:

- A cadeia de efeitos usa `vector<unique_ptr<effect>>` — cada efeito tem um dono exclusivo.
- Recursos de projeto compartilhados (presets, amostras base) usam `shared_ptr`.
- Referências para o project pai em tracks usam `weak_ptr` — evitam ciclo.

Exercícios Propostos

1. Implemente uma classe `effect_chain` que contém `vector<unique_ptr<effect>>`. Adicione métodos `add_effect(unique_ptr<effect>)` e `process(audio_buffer&)` que aplica cada efeito em sequência. Demonstre que não é possível copiar a cadeia (apenas mover).
2. Implemente um sistema de cache de amostras de áudio usando `shared_ptr`: `AudioCache` retorna `shared_ptr<audio_buffer>` para amostras carregadas; múltiplas tracks podem compartilhar a mesma amostra. Verifique que a amostra só é liberada quando a última track que a referencia é destruída.
3. Demonstre um ciclo de referência com dois objetos conectados por `shared_ptr`. Mostre que os destrutores NÃO são chamados. Corrija usando `weak_ptr` e mostre que os destrutores são chamados corretamente.
4. Por que `make_unique` e `make_shared` são preferíveis a `unique_ptr<T>(new T(...))` e `shared_ptr<T>(new T(...))`? Pesquise a questão de exception safety e a otimização de alocação no `shared_ptr`.

Capítulo 12 — Semântica de Movimento

🎯 Objetivos deste capítulo

- ▶ Compreender rvalues e lvalues e a distinção entre eles.
- ▶ Usar `std::move` para transferir recursos eficientemente.
- ▶ Implementar construtor e operador de atribuição de movimento.
- ▶ Identificar quando o compilador aplica movimento automaticamente.

12.1 O Problema: Cópias Desnecessárias

Antes de C++11, retornar um objeto grande de uma função significava copiar toda a sua memória. Com um buffer de áudio de 1 GB, isso é catastrófico. A semântica de movimento resolve isso: em vez de copiar, "roubar" os recursos do objeto de origem.

Movimento vs. cópia em retorno de função

```
// Sem movimento (C++03): retorno copiava o buffer inteiro
audio_buffer generate_silence_old(size_t n) {
    audio_buffer buf{n, 44100.0f}; // alocação
    return buf;                    // CÓPIA — alocação + cópia de n floats
}

// Com movimento (C++11): RVO ou movimento — zero alocação extra
audio_buffer generate_silence(size_t n) {
    audio_buffer buf{n, 44100.0f};
    return buf; // compilador aplica NRVO (Named Return Value Optimization)
}

// Mesmo sem NRVO, o retorno usa construtor de movimento: rouba os recursos
```

12.2 lvalues e rvalues

lvalue: tem endereço de memória, pode aparecer à esquerda de uma atribuição — variáveis nomeadas. rvalue: temporário sem nome, não tem endereço acessível ao programador — literais, resultados de expressões, retorno de funções.

lvalue vs rvalue — e `std::move`

```
int a = 42;           // a é lvalue; 42 é rvalue
int b = a + 1;        // (a + 1) é rvalue; b é lvalue

std::string s1 = "ola"; // "ola" é rvalue
std::string s2 = s1;     // s1 é lvalue — CÓPIA
std::string s3 = std::move(s1); // std::move converte s1 em rvalue — MOVIMENTO
// Após move: s1 está em estado válido mas não especificado (geralmente vazio)
```

12.3 `std::move` e Referências a rvalue (T&&)

`std::move` não move nada — ele é um cast que converte um lvalue em uma referência a rvalue (T&&), permitindo que o compilador chame o construtor/operador de movimento em vez do de cópia.

`std::move` — quando e como usar

```
void add_buffer(std::vector<audio_buffer>& collection,
               audio_buffer buf) { // parâmetro por valor
    // buf já é uma cópia ou foi movido para cá (dependendo do chamador)
    collection.push_back(std::move(buf)); // move buf para o vetor
    // buf está agora em estado vazio — não use mais
}
```

```
audio_buffer big_buf{1024 * 1024, 44100.0f};

add_buffer(collection, big_buf);           // CÓPIA de big_buf
add_buffer(collection, std::move(big_buf)); // MOVIMENTO — big_buf fica vazio
add_buffer(collection, generate_silence(1000)); // temporário: MOVIMENTO
automático
```

Cuidado:

Após usar `std::move` em um objeto, não faça suposições sobre seu estado. Ele está em "estado válido mas não especificado" — geralmente vazio/zerado, mas não garantido. Destruir ou reatribuir são as únicas operações seguras.

Exercícios Propostos

1. Adicione construtor e operador de atribuição de movimento à classe `audio_buffer` implementada anteriormente. Marque-os `noexcept`. Escreva um teste que cria um buffer grande, move-o e verifica que o original ficou vazio (`size() == 0`).
2. Escreva um benchmark simples que mede o tempo de `push_back` de 1000 `audio_buffers` de tamanho 44100 em um vector, em dois cenários: (a) `push_back(buf)` — cópia, (b) `push_back(std::move(buf))` — movimento. Reporte a diferença.
3. Por que `vector::push_back` usa movimento quando possível? Pesquise o papel de `noexcept` na decisão de realocar usando cópia vs. movimento (`strong exception guarantee`).
4. Implemente uma função `audio_buffer mix(audio_buffer left, audio_buffer right)` que recebe os parâmetros por valor, mistura os dois buffers (soma sample a sample com ganho 0.5) e retorna o resultado. Demonstre que a chamada `mix(std::move(a), std::move(b))` não copia nenhum buffer.

Capítulo 13 — Sobrecarga de Operadores

🎯 Objetivos deste capítulo

- ▶ Implementar sobrecarga de operadores aritméticos e de comparação.
- ▶ Sobrecarregar os operadores de stream << e >> para I/O idiomático.
- ▶ Seguir as convenções de membro vs. função livre para cada operador.
- ▶ Evitar armadilhas comuns na sobrecarga de operadores.

13.1 Por que Sobrecarregar Operadores?

Sobrecarga de operadores permite que tipos definidos pelo programador se comportem como tipos built-in. Em vez de `sample.add(other)`, você escreve `sample + other`. O código fica mais natural e consistente com a STL.

13.2 Regras de Ouro

- Operadores que modificam o objeto (`+=`, `-=`, `*=`) devem ser membros — recebem acesso ao estado interno.
- Operadores binários simétricos (`+`, `-`, `*`) devem ser funções livres — permitem conversão implícita em ambos os operandos.
- Operadores de stream `<<` e `>>` devem ser funções livres — o primeiro argumento é o stream, não o objeto.
- Preservar a semântica intuitiva: `a + b` não deve modificar `a` nem `b`.
- Se `operator==` é definido, defina também `operator!=` (ou use `=default` em C++20).

13.3 Exemplo: sample com Operadores

Sobrecarga completa de operadores para sample

```
// sample.hpp — operadores aritméticos e de stream
#pragma once
#include <iostream>

class sample {
public:
    explicit sample(float amplitude = 0.0f) : amp_{amplitude} {}
    float amplitude() const { return amp_; }

    /* Operador de atribuição composto (membro): modifica this */
    sample& operator+=(const sample& rhs) {
        amp_ += rhs.amp_;
        return *this;
    }
    sample& operator*=(float factor) {
        amp_ *= factor;
        return *this;
    }

    /* Operador de igualdade (C++20: auto-gera !=) */
    bool operator==(const sample&) const = default;

private:
    float amp_;
```

```
};

/* Operadores binários simétricos (funções livres, implementam via +=) */
inline sample operator+(sample lhs, const sample& rhs) {
    lhs += rhs;        // reutiliza operator+=
    return lhs;        // lhs é cópia: pode ser movido no return
}
inline sample operator*(sample lhs, float factor) {
    lhs *= factor;
    return lhs;
}
inline sample operator*(float factor, sample rhs) {
    rhs *= factor;     // comutatividade: factor * s == s * factor
    return rhs;
}

/* Operador de stream (função livre) */
inline std::ostream& operator<<(std::ostream& os, const sample& s) {
    os << "sample(" << s.amplitude() << ")";
    return os;         // permite encadeamento: cout << s1 << " " << s2
}

```

Uso idiomático dos operadores

```
// Uso natural – idêntico ao que seria com float:
sample a{0.5f}, b{0.3f};
sample c = a + b;
sample d = a * 2.0f;
sample e = 0.5f * a;

std::cout << a << " + " << b << " = " << c << "\n";
// Saída: sample(0.5) + sample(0.3) = sample(0.8)

```

Exercícios Propostos

1. Adicione à classe `audio_buffer`: (a) `operator+=(const audio_buffer&)` que soma samples correspondentes; (b) `operator*=(float)` que multiplica por um ganho; (c) operadores `+` e `*` correspondentes como funções livres; (d) `operator<<` para imprimir metadados do buffer.
2. Implemente `operator[]` para `audio_buffer` com duas versões: uma `const` (retorna `float`) e uma não-`const` (retorna `float&` permitindo modificação). O acesso fora dos limites deve lançar `std::out_of_range`.
3. Por que `operator+=` deve retornar `T&` (referência para o próprio objeto) em vez de `void` ou `T`? Demonstre o uso que isso permite e o que quebraria sem o retorno correto.
4. Pesquise o "three-way comparison operator" (spaceship operator `<=>`) de C++20. Como ele simplifica a implementação de todos os operadores de comparação? Implemente-o para `sample`.

Capítulo 14 — Testes Automatizados com Catch2

Objetivos deste capítulo

- ▶ Configurar Catch2 em um projeto CMake.
- ▶ Escrever casos de teste com TEST_CASE e REQUIRE/CHECK.
- ▶ Organizar testes com SECTION e tags.
- ▶ Testar comportamentos de exceção com REQUIRE_THROWS.
- ▶ Integrar testes ao workflow de desenvolvimento (TDD básico).

14.1 Por que Testes Automatizados?

Testes automatizados são a rede de segurança que permite refatorar código com confiança. Sem testes, cada mudança pode quebrar algo silenciosamente. Com testes: você sabe imediatamente o que quebrou, qual comportamento era esperado e qual foi obtido.

Em C++, o framework Catch2 é moderno, expressivo e fácil de integrar com CMake. É o framework adotado por padrão nesta disciplina.

14.2 Configurando Catch2 com CMake

CMakeLists.txt — integração com Catch2

```
# CMakeLists.txt — incluir Catch2 via FetchContent
include(FetchContent)
FetchContent_Declare(
    Catch2
    GIT_REPOSITORY https://github.com/catchorg/Catch2.git
    GIT_TAG         v3.5.2
)
FetchContent_MakeAvailable(Catch2)

# Alvo de teste
add_executable(tests tests/test_sample.cpp src/sample.cpp)
target_link_libraries(tests PRIVATE Catch2::Catch2WithMain)
target_include_directories(tests PRIVATE include)

# Registrar com CTest
include(CTest)
include(Catch)
catch_discover_tests(tests)
```

14.3 Primeiro Arquivo de Teste

tests/test_sample.cpp — casos de teste completos

```
// tests/test_sample.cpp
#include <catch2/catch_test_macros.hpp>
#include "sample.hpp"

/* TEST_CASE: grupo de testes relacionados */
TEST_CASE("sample: construção padrão", "[sample][construtor]") {
    sample s;
    REQUIRE(s.amplitude() == 0.0f);
}

TEST_CASE("sample: construção parametrizada", "[sample][construtor]") {
```



```

SECTION("amplitude válida") {
    sample s{0.5f};
    REQUIRE(s.amplitude() == 0.5f);
}
SECTION("amplitude no limite positivo") {
    sample s{1.0f};
    REQUIRE(s.amplitude() == 1.0f);
}
SECTION("amplitude inválida lança exceção") {
    REQUIRE_THROWS_AS(sample{1.5f}, std::domain_error);
    REQUIRE_THROWS_AS(sample{-1.1f}, std::domain_error);
}
}

TEST_CASE("sample: operadores aritméticos", "[sample][operadores]") {
    sample a{0.3f}, b{0.4f};

    SECTION("soma") {
        auto c = a + b;
        REQUIRE(c.amplitude() == Catch::Approx(0.7f)); // float: use Approx
    }
    SECTION("multiplicacao por escalar") {
        auto d = a * 2.0f;
        REQUIRE(d.amplitude() == Catch::Approx(0.6f));
    }
    SECTION("operador += modifica in-place") {
        a += b;
        REQUIRE(a.amplitude() == Catch::Approx(0.7f));
    }
}

```

Executar os testes

```

# Executar os testes:
cd build
cmake --build .
ctest --output-on-failure
# ou diretamente:
./tests

```



Boa prática:

Execute os testes a cada commit. Configure um atalho no terminal: `alias t="cmake --build build && ctest --test-dir build --output-on-failure"`. Com GitHub Actions (Capítulo 5), os testes rodam automaticamente em cada push.

14.4 Testando Comportamentos de Exceção e Edge Cases

Testes para audio_buffer

```

TEST_CASE("audio_buffer: acesso por índice", "[audio_buffer]") {
    audio_buffer buf{10, 44100.0f};

    SECTION("acesso dentro dos limites") {
        buf[0] = 0.5f;
        REQUIRE(buf[0] == Catch::Approx(0.5f));
    }
}

```

```

    SECTION("acesso fora dos limites lança out_of_range") {
        REQUIRE_THROWS_AS(buf[10], std::out_of_range);
        REQUIRE_THROWS_AS(buf[100], std::out_of_range);
    }
    SECTION("buffer vazio: size zero") {
        audio_buffer empty;
        REQUIRE(empty.size() == 0);
    }
}

TEST_CASE("audio_buffer: semantica de movimento", "[audio_buffer][movimento]")
{
    audio_buffer src{1024, 44100.0f};
    size_t orig_size = src.size();

    audio_buffer dst = std::move(src);
    REQUIRE(dst.size() == orig_size);
    REQUIRE(src.size() == 0); // src foi esvaziado pelo movimento
}

```

Exercícios Propostos

1. Configure Catch2 via FetchContent no projeto Sintonia e escreva um arquivo de testes para a classe track: teste construção, add_sample, size(), operator[] e o comportamento quando um índice inválido é acessado.
2. Implemente TDD (Test-Driven Development) para uma classe audio_effects::normalize(audio_buffer&) que escala todos os samples para que o máximo absoluto seja 1.0. Escreva os testes primeiro, depois implemente.
3. Escreva testes que verificam a semântica de cópia e movimento para audio_buffer: após copiar, modifique a cópia e verifique que o original não foi alterado. Após mover, verifique que o original ficou vazio.
4. Configure o GitHub Actions para compilar e rodar os testes automaticamente em cada push. Crie um arquivo .github/workflows/ci.yml que instala as dependências, compila com CMake e roda ctest.

Capítulo 15 — Herança Simples em C++

🎯 Objetivos deste capítulo

- ▶ Modelar hierarquias de classes com herança pública.
- ▶ Distinguir herança (is-a) de composição (has-a).
- ▶ Compreender o encadeamento de construtores e destrutores na hierarquia.
- ▶ Usar protected corretamente.

15.1 Herança como Modelagem de Domínio

Herança modela a relação "é um" (is-a): um Reverb é um Effect; um GradStudent é um Student. Antes de usar herança, pergunte: "essa relação realmente é is-a, ou é has-a?" Abusar de herança onde composição seria mais apropriado é um dos erros mais comuns em POO.

Use herança quando...	Use composição quando...
A relação é genuinamente is-a	A relação é has-a ou uses-a
A classe derivada pode ser usada no lugar da base (Liskov)	A funcionalidade é implementada em termos de outra sem expor interface
Você quer polimorfismo dinâmico (virtual)	Você quer flexibilidade: trocar implementação em runtime
A hierarquia é estável e bem compreendida	A hierarquia cresceria demais ou seria frágil

15.2 Sintaxe e Tipos de Herança em C++

Sintaxe de herança — três formas

```
// Herança pública (is-a): mantém interface da base
class derived : public base { /* ... */ };

// Herança protegida: interface da base vira protected
class derived : protected base { /* ... */ };

// Herança privada (implementada em termos de): interface da base vira private
class derived : private base { /* ... */ };
```

Na prática, herança pública é a mais comum (modelagem is-a com polimorfismo). Herança privada é uma alternativa à composição quando precisamos de acesso à implementação interna da base (raro).

15.3 Hierarquia de Efeitos — Sintonia v1.0

effect.hpp — classe base com destrutor virtual

```
// effect.hpp — classe base abstrata
#pragma once
#include "audio_buffer.hpp"
```

```
class effect {
public:
    /* Construtor com nome do efeito */
    explicit effect(std::string name) : name_{std::move(name)} {}

    /* Destrutor virtual: OBRIGATÓRIO em classes base polimórficas */
    virtual ~effect() = default;

    /* Interface pública */
    virtual void process(audio_buffer& buf) = 0; // pura: subclasses DEVEM
    implementar

    const std::string& name() const { return name_; }
    bool enabled() const { return enabled_; }
    void set_enabled(bool e) { enabled_ = e; }

protected:
    /* Método auxiliar: subclasses podem usar, código externo não */
    void log_processing() const;

private:
    std::string name_;
    bool enabled_ = true;
};
```

gain_effect.hpp/.cpp — classe derivada com override

```
// gain_effect.hpp — classe derivada
#pragma once
#include "effect.hpp"

class gain_effect : public effect {
public:
    explicit gain_effect(float factor)
        : effect{"Gain"}, // inicializa BASE primeiro
        factor_{factor}
    {}

    /* override: garante que estamos sobrescrevendo um virtual da base */
    void process(audio_buffer& buf) override;

    float factor() const { return factor_; }

private:
    float factor_;
};

// gain_effect.cpp
#include "gain_effect.hpp"
void gain_effect::process(audio_buffer& buf) {
    for (size_t i = 0; i < buf.size(); ++i)
        buf[i] *= factor_;
}
```

Cuidado:

SEMPRE declare o destrutor de classes base como virtual.

Sem virtual `~effect()`, deletar um `gain_effect*` via ponteiro base (`effect*`) chama apenas o destrutor de `effect` — o destrutor de `gain_effect` NÃO é chamado. Isso causa vazamento de recursos.

15.4 Construtores e Destrutores na Hierarquia

Ordem de construção e destruição na hierarquia

```
// Ordem de construção: BASE primeiro, depois DERIVADA
gain_effect g{0.5f};
// 1. effect::effect("Gain") é chamado
// 2. gain_effect::gain_effect(0.5f) completa

// Ordem de destruição: INVERSA — DERIVADA primeiro, depois BASE
// gain_effect::~~gain_effect()
// effect::~~effect()
```

DIAGRAMA UML

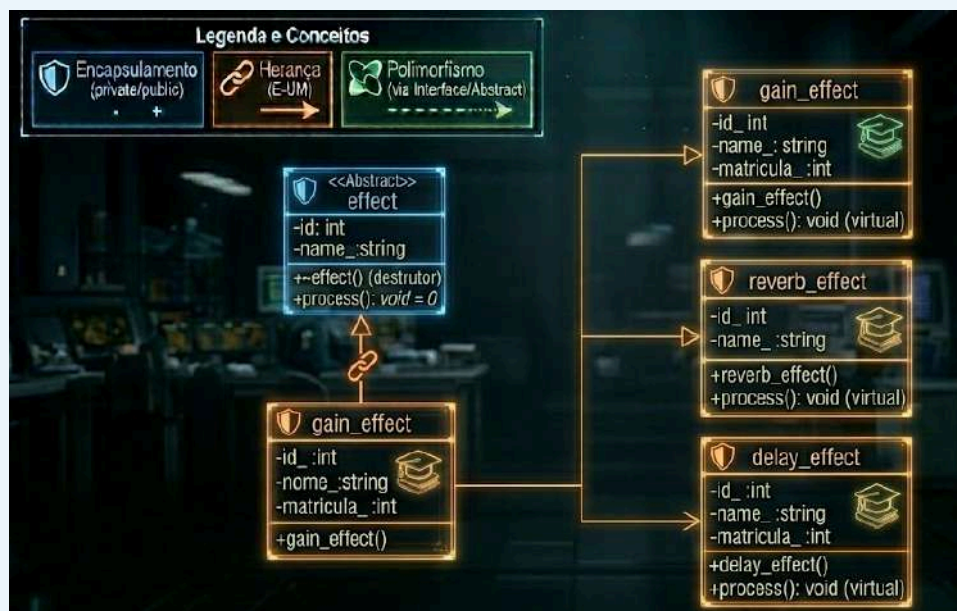


Diagrama UML: hierarquia de efeitos — `effect` (abstrata) ← `gain_effect`, `reverb_effect`, `delay_effect`.
Mostrar herança com seta fechada, método `process()` virtual puro.

Exercícios Propostos

1. Implemente `reverb_effect : public effect` com parâmetro `room_size` (float 0..1) que aplica um eco simplificado no `audio_buffer`. Herde corretamente de `effect` e use `override`.
2. Implemente `delay_effect : public effect` com parâmetros `delay_samples` (size_t) e `feedback` (float). O delay adiciona uma versão atrasada do sinal ao sinal original.
3. Crie uma `effect_chain` que contém `vector<unique_ptr<effect>>` e um método `process(audio_buffer&)` que aplica cada efeito em sequência, pulando efeitos com `enabled() == false`. Escreva testes Catch2.

4. Explique por que remover "virtual" do destrutor de `effect` levaria a vazamento de recursos. Demonstre com código: crie um `unique_ptr<effect>` apontando para um `gain_effect`, destrua-o e mostre com mensagens nos destrutores que o destrutor correto é (ou não é) chamado.

Capítulo 16 — Herança Múltipla e o Problema do Diamante

🎯 Objetivos deste capítulo

- ▶ Usar herança múltipla de interfaces sem ambiguidades.
- ▶ Compreender o problema do diamante e suas consequências.
- ▶ Resolver o diamante com herança virtual.
- ▶ Contrastar com a solução de Java/C# (interfaces únicas, herança simples de classes).

16.1 Herança Múltipla: Capacidades e Riscos

C++ permite que uma classe herde de múltiplas classes base simultaneamente. Isso é poderoso — mas exige cuidado. O caso mais seguro é herdar de múltiplas classes puramente abstratas (interfaces), que é o que Java e C# fazem com "implementar múltiplas interfaces".

Herança múltipla segura — de interfaces puras

```
// Caso seguro: herança de múltiplas interfaces (classes abstratas puras)
class recordable {
public:
    virtual void start_recording() = 0;
    virtual void stop_recording() = 0;
    virtual ~recordable() = default;
};

class playable {
public:
    virtual void play() = 0;
    virtual void pause() = 0;
    virtual ~playable() = default;
};

// Looper implementa ambas as interfaces — sem diamante
class looper : public recordable, public playable {
public:
    void start_recording() override { /* ... */ }
    void stop_recording() override { /* ... */ }
    void play() override { /* ... */ }
    void pause() override { /* ... */ }
};
```

16.2 O Problema do Diamante

O problema do diamante ocorre quando uma classe herda indiretamente de outra classe de duas rotas diferentes. Isso cria duas cópias dos membros da classe ancestral — causando ambiguidade e desperdício de memória.

Diamante sem herança virtual — o problema

```
// Hierarquia em diamante — SEM herança virtual (PROBLEMÁTICO)
class sound_source {
public:
    std::string name_; // este membro existirá DUAS vezes em looper!
    virtual void generate(audio_buffer&) = 0;
    virtual ~sound_source() = default;
};
```

```

};

class oscillator : public sound_source {
    /* gera onda sintética */ };

class sampler : public sound_source {
    /* reproduz amostra gravada */ };

// ✗ PROBLEMA: looper herda sound_source via oscillator E via sampler
// Resultado: DUAS cópias de sound_source::name_ em looper
class looper : public oscillator, public sampler {
    // looper.name_ é ambíguo: qual das duas cópias?
    // looper.oscillator::sound_source::name_ ou
    // looper.sampler::sound_source::name_?
};

```

DIAGRAMA UML

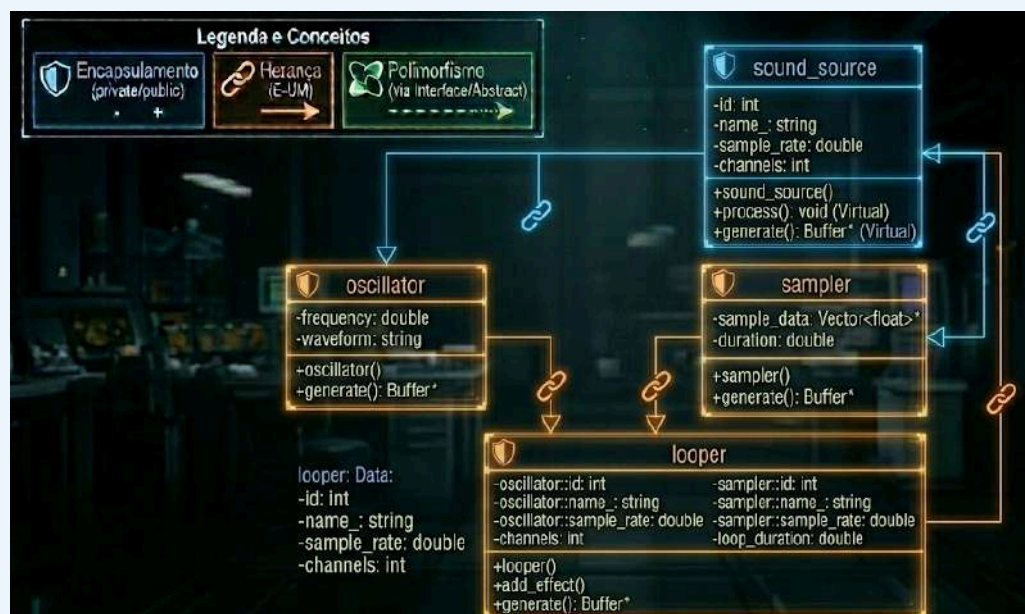


Diagrama UML do diamante: *sound_source* no topo; *oscillator* e *sampler* herdam de *sound_source*; *looper* herda de *oscillator* e *sampler*. Mostrar as DUAS cópias de *sound_source* em *looper* sem virtual.

16.3 Resolvendo com Herança Virtual

Com herança virtual, a classe ancestral compartilhada tem exatamente uma instância em toda a hierarquia — não importa quantas rotas de herança existam.

Herança virtual — resolução do diamante

```

// ✓ SOLUÇÃO: herança virtual garante UMA cópia de sound_source
class sound_source {
public:
    std::string name_;
    virtual void generate(audio_buffer&) = 0;
    virtual ~sound_source() = default;
};

/* "virtual" aqui: sound_source será compartilhada */
class oscillator : public virtual sound_source {
    void generate(audio_buffer&) override;

```



```

};

class sampler : public virtual sound_source {
    void generate(audio_buffer&) override;
};

/* looper herda UMA instância de sound_source */
class looper : public oscillator, public sampler {
public:
    explicit looper(std::string name) {
        /* Com herança virtual, a classe mais derivada inicializa a base virtual */
        name_ = std::move(name);
    }
    /* Deve resolver qual generate() usar — ou declarar sua própria */
    void generate(audio_buffer& buf) override {
        oscillator::generate(buf); // usa explicitamente a versão do oscillator
    }
};

// Uso:
looper lp{"meu_looper"};
std::cout << lp.name_ << "\n"; // sem ambiguidade agora

```

⚠ Cuidado:

Com herança virtual, a classe mais derivada (looper) é responsável por inicializar a classe base virtual (sound_source), mesmo que não herde diretamente dela. Herança virtual adiciona um overhead de indireção (vptra extra). Use apenas quando o diamante realmente ocorrer.

16.4 Contraste com Java/C#

Java e C# eliminam o problema simplesmente proibindo herança múltipla de classes concretas. Em compensação, permitem implementar múltiplas interfaces (equivalentes às classes abstratas puras de C++). O resultado é uma solução mais restrita mas mais simples — a maioria dos usos práticos de herança múltipla em C++ envolve apenas interfaces puras, que não têm o problema do diamante.

Exercícios Propostos

1. Implemente a hierarquia `sound_source` → `oscillator/sampler` → `looper` com herança virtual corretamente. Adicione uma mensagem nos construtores de cada classe e observe a ordem de construção.
2. Demonstre o problema do diamante sem herança virtual: crie a hierarquia, instancie `looper` e tente acessar `name_` — mostre o erro de compilação por ambiguidade.
3. Implemente `sinusoidal_oscillator : public oscillator` que gera uma onda senoidal no processo de `generate()`. Adicione `file_sampler : public sampler` que lê um arquivo WAV. Crie um `looper_synth : public sinusoidal_oscillator, public file_sampler`.
4. Pesquise o "Mixin" pattern em C++ usando herança múltipla de classes com implementação. Como mixins diferem do diamante? Dê um exemplo de mixin que não causa problema.

Capítulo 17 — Funções Virtuais e Classes Abstratas

🎯 Objetivos deste capítulo

- ▶ Usar virtual, override e final corretamente.
- ▶ Declarar classes abstratas com funções virtuais puras.
- ▶ Compreender o mecanismo interno das vtables.
- ▶ Evitar chamadas a funções virtuais em construtores e destrutores.

17.1 A Necessidade de Funções Virtuais

Sem virtual, a chamada a um método é resolvida em tempo de compilação pelo tipo estático do ponteiro/referência (ligação estática). Com virtual, a chamada é resolvida em tempo de execução pelo tipo dinâmico do objeto (ligação dinâmica). Isso é o que permite polimorfismo.

Ligação estática vs. dinâmica

```
// Sem virtual – ligação ESTÁTICA (comportamento surpresa):
class shape {
public:
    void draw() { std::cout << "Shape\n"; } // NÃO virtual
};
class circle : public shape {
public:
    void draw() { std::cout << "Circle\n"; } // oculta, não sobrescreve
};

circle c;
shape& ref = c;
ref.draw(); // imprime "Shape" – chama shape::draw() pelo tipo ESTÁTICO

// Com virtual – ligação DINÂMICA (comportamento correto):
class shape2 {
public:
    virtual void draw() { std::cout << "Shape2\n"; }
    virtual ~shape2() = default;
};
class circle2 : public shape2 {
public:
    void draw() override { std::cout << "Circle2\n"; }
};

circle2 c2;
shape2& ref2 = c2;
ref2.draw(); // imprime "Circle2" – chama circle2::draw() pelo tipo DINÂMICO
```

17.2 override e final

override e final foram introduzidos em C++11. override garante em tempo de compilação que a função realmente sobrescreve um virtual da base — pega erros de assinatura. final impede que a função seja sobrescrita em classes derivadas (ou que a classe inteira seja usada como base).

override e final — segurança e intenção explícita

```

class effect {
public:
    virtual void process(audio_buffer& buf) = 0;
    virtual ~effect() = default;
};

class gain_effect : public effect {
public:
    void process(audio_buffer& buf) override; // OK: sobrescreve o virtual
    // void process(audio_buffer buf) override; // ERRO: assinatura diferente
};

class limiter : public gain_effect {
public:
    void process(audio_buffer& buf) override final; // não pode ser
    sobrescrito
};

class optimized_limiter final : public limiter { // não pode ser subclasse
    /* ... */
};

```

Boa prática:

Sempre use override em toda função virtual de uma classe derivada. O compilador detecta imediatamente se você errou a assinatura — um erro comum que sem override se manifesta como "método não está sendo chamado" em runtime.

17.3 Funções Virtuais Puras e Classes Abstratas

Uma função virtual pura (= 0) significa que a classe base não fornece implementação — qualquer subclasse concreta DEVE implementar. Uma classe com pelo menos uma função virtual pura é abstrata: não pode ser instanciada.

Classe abstrata — effect como interface com comportamento parcial

```

/* effect é abstrata: não pode ser instanciada */
class effect {
public:
    virtual void process(audio_buffer& buf) = 0; // pura
    virtual std::string name() const = 0;      // pura
    virtual ~effect() = default;

    /* Método concreto (não puro) pode existir em classes abstratas */
    bool enabled() const { return enabled_; }
    void set_enabled(bool e) { enabled_ = e; }

private:
    bool enabled_ = true;
};

// effect e{}; // ERRO: instânciação de classe abstrata
gain_effect g{0.5f}; // OK: implementa todas as puras

```

17.4 Mecanismo Interno: vtable e vptr

Para cada classe com ao menos uma função virtual, o compilador gera uma vtable (tabela de funções virtuais - Figura 5) — um array de ponteiros para as implementações corretas de cada função virtual. Cada objeto da classe tem um vptr (ponteiro para a vtable) oculto.

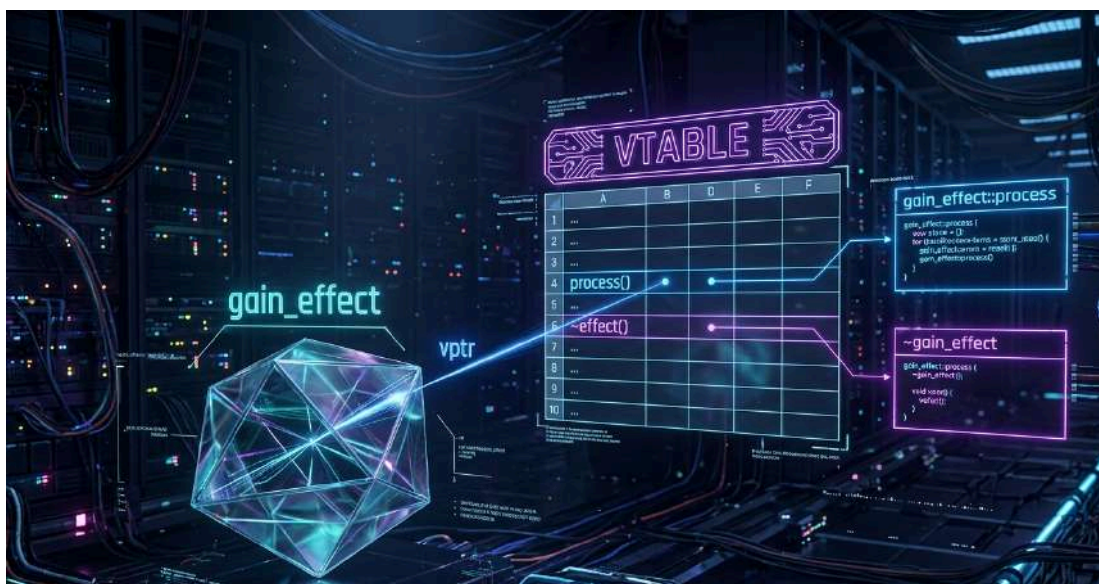


Figura 5. Um detalhado esquema 3D do funcionamento de Vtables e dispatch dinâmico, mostrando como a tabela vptr (virtual pointer) de um objeto é usada para buscar e chamar as implementações de funções virtuais em tempo de execução.

Quando você chama `ref.process(buf)` via referência/ponteiro, o compilador emite: (1) carrega o vptr do objeto, (2) indexa a vtable pelo slot de `process()`, (3) chama a função encontrada. Esse é o "overhead" do despacho virtual — uma indireção a mais em relação a uma chamada direta.

DIAGRAMA UML

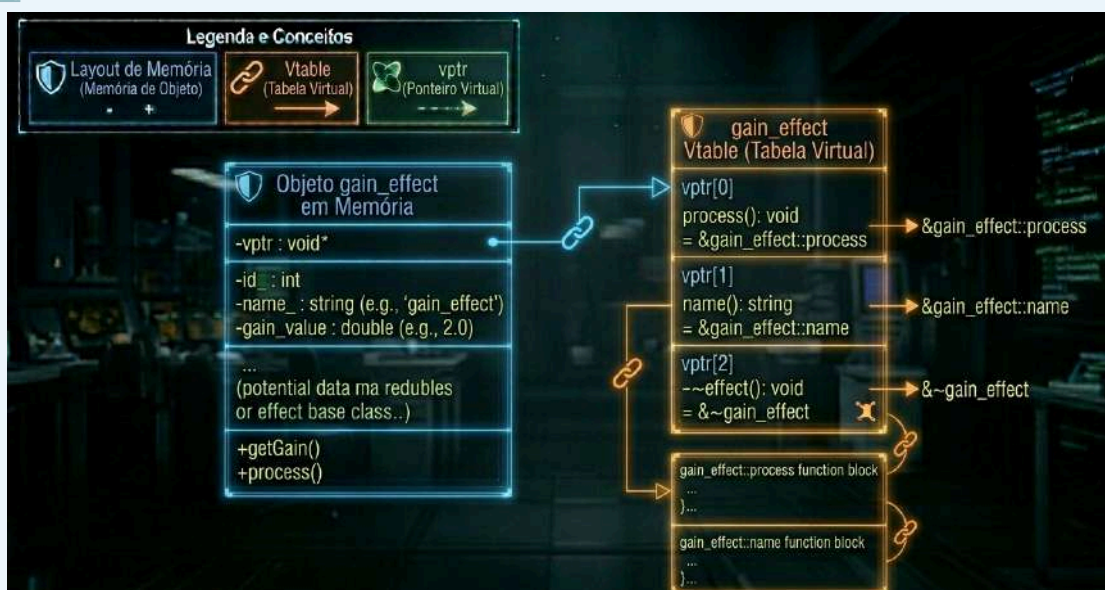


Diagrama: objeto `gain_effect` em memória com `vptr` apontando para `gain_effect` vtable; `gain_effect` vtable com slots: `process` → `gain_effect::process`, `name` → `gain_effect::name`, `~effect` → `~gain_effect`.

⚠ Cuidado:

Nunca chame funções virtuais em construtores ou destrutores. Durante a construção de `gain_effect`, o `vptr` ainda aponta para a `vtable` de `effect` (ou pior, é inválido). A função virtual chamada será a da classe que está sendo construída no momento — não a da classe derivada que você esperava.

Exercícios Propostos

1. Implemente uma hierarquia de instrumentos musicais: `instrument` (abstrata, virtual `produce_sound(audio_buffer&)`), `guitar` : public `instrument`, `piano` : public `instrument`. Crie um `vector<unique_ptr<instrument>>` e processe-o polimorficamente.
2. Demonstre o bug de chamar uma função virtual no construtor: crie uma classe base com construtor que chama um virtual e uma derivada que o sobrescreve. Mostre qual versão é chamada durante a construção.
3. Adicione o método virtual `std::string describe() const` à classe `effect`. Forneça uma implementação padrão na base que retorna apenas o nome. Sobrescreva em `gain_effect` para incluir o fator. Mostre que a versão correta é chamada via ponteiro base.
4. Meça o custo de despacho virtual vs. chamada direta: escreva um benchmark que chama `process()` em 1 milhão de samples via (a) ponteiro base (virtual), (b) diretamente no objeto concreto. Qual a diferença percentual?

Capítulo 18 — Polimorfismo Dinâmico, RTTI e `dynamic_cast`

🎯 Objetivos deste capítulo

- ▶ Compreender polimorfismo dinâmico via vtable.
- ▶ Usar `dynamic_cast` para downcasting seguro.
- ▶ Entender RTTI (`typeid`, `type_info`) e seus limites.
- ▶ Identificar quando `dynamic_cast` indica problema de design.

18.1 Polimorfismo Dinâmico em Ação

Polimorfismo dinâmico permite processar objetos de tipos diferentes através de uma interface comum. O tipo real é irrelevante para o código cliente — ele apenas chama o método e o objeto correto responde.

Polimorfismo dinâmico — cadeia de efeitos

```
#include <memory>
#include <vector>
#include <iostream>

// Código cliente: não sabe qual efeito está processando
void process_chain(std::vector<std::unique_ptr<effect>>& chain,
                  audio_buffer& buf) {
    for (auto& eff : chain) {
        if (eff->enabled())
            eff->process(buf); // despacho dinâmico: chama o tipo certo
    }
}

// Criação polimórfica:
std::vector<std::unique_ptr<effect>> chain;
chain.push_back(std::make_unique<gain_effect>(0.8f));
chain.push_back(std::make_unique<reverb_effect>(0.3f));
chain.push_back(std::make_unique<delay_effect>(4410, 0.5f));

audio_buffer buf{44100, 44100.0f};
process_chain(chain, buf); // cada efeito processa à sua maneira
```

18.2 `dynamic_cast` — Downcasting Seguro

Downcast é a operação de converter um ponteiro/referência para um tipo base em um tipo derivado. `dynamic_cast` verifica em runtime se o objeto realmente é do tipo alvo — retorna `nullptr` (ponteiros) ou lança `std::bad_cast` (referências) se não for.

`dynamic_cast` — verificação em runtime

```
// dynamic_cast com ponteiro — retorna nullptr se falhar
effect* ptr = chain[0].get(); // ponteiro para base

if (auto* g = dynamic_cast<gain_effect*>(ptr)) {
    // g é gain_effect* — podemos acessar membros específicos
    std::cout << "Fator de ganho: " << g->factor() << "\n";
} else {
    std::cout << "Não é um gain_effect\n";
}
```

```
// dynamic_cast com referência — lança std::bad_cast se falhar
try {
    auto& g = dynamic_cast<gain_effect*>(*ptr);
    // uso de g
} catch (const std::bad_cast& e) {
    std::cerr << "Cast falhou: " << e.what() << "\n";
}
```

⚠ Cuidado:

Usar `dynamic_cast` frequentemente é um sintoma de design ruim — indica que o código cliente sabe demais sobre os tipos concretos.

Se você se pega fazendo `if (dynamic_cast<tipo_a>()) ... else if (dynamic_cast<tipo_b>()) ...`, considere redesenhar usando o padrão Visitor ou adicionando o método necessário na interface base.

18.3 RTTI — typeid e type_info

RTTI (Run-Time Type Information) fornece informação sobre o tipo real de um objeto em runtime. `typeid(expr)` retorna uma referência para um objeto `type_info` que pode ser comparado com `==` e tem um método `name()`.

RTTI com typeid

```
#include <typeinfo>

effect* ptr = chain[0].get();

// typeid em ponteiro não-nulo: tipo dinâmico do objeto apontado
std::cout << typeid(*ptr).name() << "\n"; // ex: "11gain_effect" (mangled)

// Comparação de tipos
if (typeid(*ptr) == typeid(gain_effect))
    std::cout << "É gain_effect\n";

// ATENÇÃO: typeid(*ptr) requer que o ponteiro não seja nulo
// e que a classe tenha pelo menos uma função virtual (RTTI habilitado)
```

Em geral, prefira `dynamic_cast` a comparações com `typeid` — o primeiro funciona com toda a hierarquia (subtipo), o segundo é apenas igualdade exata de tipo.

Exercícios Propostos

1. Implemente uma função `void dump_chain(const vector<unique_ptr<effect>>&)` que usa `dynamic_cast` para identificar o tipo de cada efeito e imprime seus parâmetros específicos (fator do gain, room_size do reverb, etc.).
2. Refatore `dump_chain` para não usar `dynamic_cast`: adicione um método virtual `describe()` const em `effect` que retorna uma string com os parâmetros. Compare as duas versões em termos de coesão e acoplamento.

- 3.** Pesquise o padrão Visitor como alternativa ao `dynamic_cast` para operações específicas de tipo. Implemente um visitor `AudioAnalyzer` que visita cada efeito da cadeia e coleta estatísticas.
- 4.** Por que `dynamic_cast` só funciona com classes polimórficas (que têm pelo menos uma função virtual)? O que acontece se você tentar `dynamic_cast` em uma classe sem virtual?

UNIDADE III

Abstração Genérica, Engenharia OO e Ecossistema

Templates, SOLID, padrões de projeto, concorrência, serialização e Qt

Capítulo 19 — Templates e Polimorfismo Estático

🎯 Objetivos deste capítulo

- ▶ Escrever e usar templates de função e de classe.
- ▶ Compreender a diferença entre polimorfismo estático (templates) e dinâmico (virtual).
- ▶ Introduzir o Curiously Recurring Template Pattern (CRTP).
- ▶ Entender dedução de tipo e especialização de templates.

19.1 O que São Templates?

Templates permitem escrever código genérico — parametrizado por tipo — que é instanciado pelo compilador para cada tipo concreto usado (Figura 6).

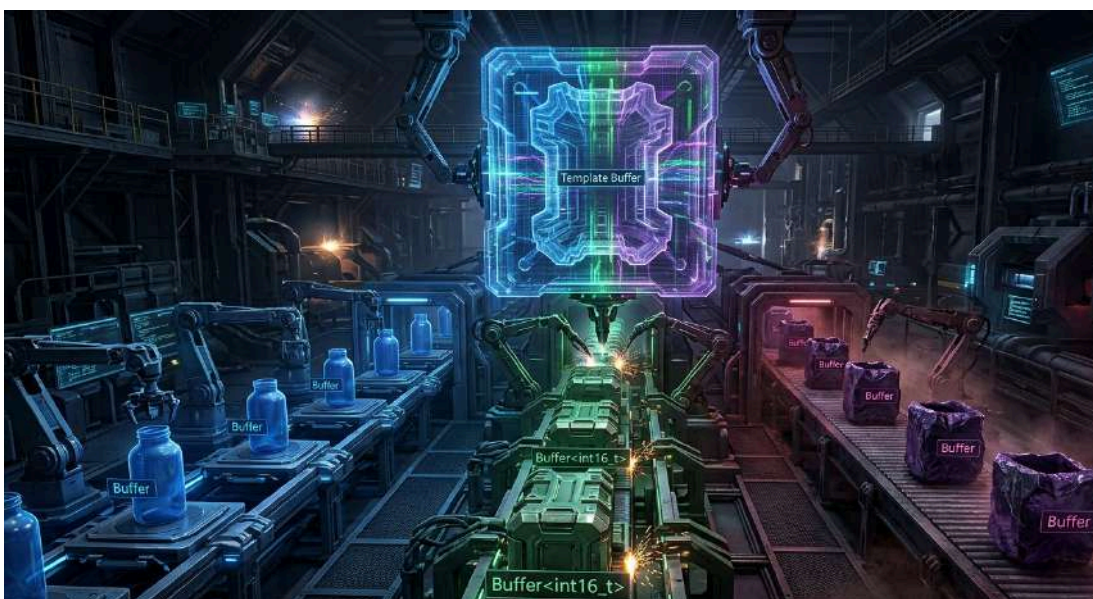


Figura 6. Uma fábrica futurista onde o "Template Buffer" (um blueprint genérico) é usado como molde para criar diferentes tipos de contêineres: vidro (básico), metal (otimizado para int16) e obsidiana (pesado). Todos compartilhando o mesmo design fundamental.

Ao contrário do polimorfismo virtual (runtime), templates resolvem o tipo em tempo de compilação — polimorfismo estático. O resultado: sem overhead de vtable, código otimizável.

19.2 Templates de Função

Templates de função — parametrização por tipo

```
// Template de função: funciona com qualquer tipo com operator<
template <typename T>
T max_of(T a, T b) {
    return (a > b) ? a : b;
}

// O compilador gera versões específicas para cada tipo usado:
auto x = max_of(3, 7);           // T = int
auto y = max_of(3.14, 2.71);    // T = double
auto z = max_of(std::string{"a"}, std::string{"b"}); // T = std::string
```

```
// Template com múltiplos parâmetros de tipo:
template <typename From, typename To>
To convert(From value) {
    return static_cast<To>(value);
}
auto f = convert<int, float>(42);
```

19.3 Templates de Classe — Buffer<T>

buffer<T> — template de classe genérico

```
// buffer<T>: versão genérica do audio_buffer — Sintonia v2.0
template <typename T>
class buffer {
public:
    explicit buffer(size_t size, T init = T{})
        : data_(size, init), sample_rate_{44100.0f} {}

    T& operator[](size_t i) { return data_.at(i); }
    const T& operator[](size_t i) const { return data_.at(i); }

    size_t size() const { return data_.size(); }
    float sample_rate() const { return sample_rate_; }

    /* Algoritmo genérico: aplica função a cada sample */
    template <typename Func>
    void apply(Func f) {
        for (auto& s : data_) s = f(s);
    }

private:
    std::vector<T> data_;
    float sample_rate_;
};

// Instanciações diferentes:
buffer<int16_t> pcm{44100}; // áudio PCM 16-bit
buffer<float> fp32{44100}; // áudio float 32-bit
buffer<double> fp64{44100}; // áudio double 64-bit

// Uso do método genérico apply:
fp32.apply([](float s) { return s * 0.5f; }); // ganho 0.5
```

19.4 Polimorfismo Estático vs. Dinâmico

Característica	Templates (estático)	Virtual (dinâmico)
Quando é resolvido?	Compilação	Runtime
Overhead por chamada?	Nenhum (inlining possível)	Indireção via vtable
Código binário?	Instância separada por tipo (code bloat)	Uma implementação, despacho por vptr
Tipos em coleção heterogênea?	Não (todos do mesmo tipo)	Sim (ponteiros para base)

Característica	Templates (estático)	Virtual (dinâmico)
Extensão em plugins?	Não (tipos fixos em compilação)	Sim (tipos adicionados em runtime)
Erros detectados?	Compilação (mensagens podem ser crípticas)	Runtime (dynamic_cast, bad_cast)

19.5 CRTP — Curiously Recurring Template Pattern

CRTP é uma técnica de polimorfismo estático onde uma classe herda de um template parametrizado por ela mesma. Permite implementar "interfaces" sem vtable — o custo de resolução é zero.

CRTP — polimorfismo sem vtable

```
// CRTP: classe base parametrizada pela derivada
template <typename Derived>
class effect_base {
public:
    /* process() chama a implementação da classe derivada — sem virtual */
    void process(buffer<float>& buf) {
        static_cast<Derived*>(this)->process_impl(buf);
    }
    std::string name() const {
        return static_cast<const Derived*>(this)->name_impl();
    }
};

class gain_static : public effect_base<gain_static> {
    float factor_;
public:
    explicit gain_static(float f) : factor_{f} {}
    void process_impl(buffer<float>& buf) {
        buf.apply([this](float s) { return s * factor_; });
    }
    std::string name_impl() const { return "GainStatic"; }
};

// Uso com template: resolvido em compilação, sem vtable
template <typename Effect>
void apply_effect(Effect& eff, buffer<float>& buf) {
    eff.process(buf); // sem despacho virtual — inlineável
}
```

Exercícios Propostos

1. Implemente uma função template `normalize(buffer<T>&)` que escala os samples para que o máximo absoluto seja 1.0. Teste com `buffer<float>` e `buffer<double>`.
2. Adicione ao template `buffer<T>` um método template `<typename Func> buffer<T> transformed(Func f) const` que retorna um novo buffer com a função aplicada.
3. Use CRTP para implementar um logging mixin que imprime o nome do efeito antes e depois de `process()`. Aplique ao `gain_static` e `reverb_static`.

4. Compare o assembly gerado (via Compiler Explorer/godbolt.org) para a mesma operação implementada com virtual e com template CRTP. Identifique as diferenças.

Capítulo 20 — Concepts (C++20) e Ranges

🎯 Objetivos deste capítulo

- ▶ Usar concepts para restringir parâmetros de template com mensagens de erro legíveis.
- ▶ Definir concepts customizados com requires.
- ▶ Usar ranges e views para expressão funcional sobre sequências.

20.1 O Problema dos Templates sem Restrições

Templates de C++17 aceitam qualquer tipo — erros de tipo só são detectados ao tentar usar um tipo incompatível, gerando mensagens de erro dificilmente legíveis. Concepts resolvem isso declarando explicitamente o que um tipo deve suportar.

Concepts — restrições explícitas em templates

```
// C++17: erro críptico se T não suportar operator<
template <typename T>
T max_of(T a, T b) { return (a > b) ? a : b; }

// C++20: concept Comparable restringe T com mensagem clara
#include <concepts>

template <typename T>
concept numeric = std::is_arithmetic_v<T>;

template <numeric T>
T max_of(T a, T b) { return (a > b) ? a : b; }

// Concept customizado: tipo "processável"
template <typename T>
concept audio_sample = requires(T a, T b, float f) {
    { a + b } -> std::convertible_to<T>;
    { a * f } -> std::convertible_to<T>;
    { a > b } -> std::convertible_to<bool>;
};

template <audio_sample T>
T mix(T a, T b) { return (a + b) * 0.5f; }
```

20.2 Ranges — Programação Funcional sobre Sequências

Ranges e views — composição lazy

```
#include <ranges>
#include <vector>
#include <iostream>
#include <algorithm>

std::vector<float> samples = {0.1f, -0.5f, 0.8f, -0.2f, 0.4f};

// Ranges: composição de views (avaliação lazy)
auto positive = samples
    | std::views::filter([](float s) { return s > 0.0f; })
    | std::views::transform([](float s) { return s * 2.0f; });

for (float s : positive)
    std::cout << s << " "; // 0.2 1.6 0.8
```

```
// Sem ranges (verbose):  
std::vector<float> result;  
for (float s : samples)  
    if (s > 0.0f) result.push_back(s * 2.0f);
```

Exercícios Propostos

1. Reescreva `buffer<T>::apply()` usando `std::ranges::transform` em vez do loop manual. Verifique que o comportamento é idêntico.
2. Crie um concept `processable_by<Effect, T>` que verifica se `Effect` tem um método `process_impl(buffer<T>&)`. Use-o para restringir o template `apply_effect`.
3. Use `std::views::zip` (C++23) ou implemente manualmente para misturar dois buffers usando ranges. Compare a legibilidade com a versão de loop manual.

Capítulo 21 — Tratamento de Erros

🎯 Objetivos deste capítulo

- ▶ Usar exceções com hierarquias de `std::exception`.
- ▶ Aplicar as garantias de exceção: básica, forte e `noexcept`.
- ▶ Usar `std::optional` para resultados opcionais sem exceção.
- ▶ Usar `std::variant` para resultados que podem ser erro ou valor.

21.1 Exceções em C++

Exceções propagam erros automaticamente pela pilha de chamadas até um handler que saiba tratá-los. Ao contrário de códigos de retorno, erros não podem ser ignorados silenciosamente — se não forem tratados, terminam o programa com mensagem clara.

Hierarquia de exceções para o Sintonia

```
// Hierarquia de exceções customizadas
#include <stdexcept>

/* Base para erros do Sintonia */
class sintonia_error : public std::runtime_error {
public:
    explicit sintonia_error(const std::string& msg)
        : std::runtime_error{"[Sintonia] " + msg} {}
};

class file_error : public sintonia_error {
public: using sintonia_error::sintonia_error; };

class format_error : public sintonia_error {
public: using sintonia_error::sintonia_error; };

// Uso:
audio_buffer load_wav(const std::string& path) {
    std::ifstream f{path, std::ios::binary};
    if (!f) throw file_error{"arquivo nao encontrado: " + path};
    if (!check_riff(f)) throw format_error{"formato WAV inválido: " + path};
    return parse_wav(f);
}

// Handler:
try {
    auto buf = load_wav("input.wav");
    process(buf);
} catch (const file_error& e) {
    std::cerr << "Erro de arquivo: " << e.what() << "\n";
} catch (const sintonia_error& e) {
    std::cerr << "Erro do Sintonia: " << e.what() << "\n";
} catch (const std::exception& e) {
    std::cerr << "Erro inesperado: " << e.what() << "\n";
}
```

21.2 `std::optional` — Resultado Ausente sem Exceção

`std::optional<T>` representa um valor que pode ou não estar presente — alternativa leve a exceções para operações que "podem falhar" sem ser erros excepcionais (como buscar em um mapa).

`std::optional` — resultado opcional

```
#include <optional>

// Retorna o buffer se encontrado, std::nullopt se não
std::optional<audio_buffer> find_cached(const std::string& key) {
    auto it = cache_.find(key);
    if (it == cache_.end()) return std::nullopt;
    return it->second;
}

// Uso — força o chamador a verificar:
auto result = find_cached("drums.wav");
if (result) {
    process(*result); // desreferência segura
} else {
    std::cout << "Não está em cache\n";
}

// Ou com value_or:
auto buf = find_cached("drums.wav").value_or(generate_silence(1024));
```

21.3 `std::variant` — Resultado ou Erro

`std::variant` — resultado ou erro

```
#include <variant>
#include <string>

// Resultado pode ser buffer OU mensagem de erro
using load_result = std::variant<audio_buffer, std::string>;

load_result try_load(const std::string& path) {
    std::ifstream f{path, std::ios::binary};
    if (!f) return std::string{"arquivo nao encontrado: " + path};
    return parse_wav(f); // retorna audio_buffer
}

// Uso com std::visit:
auto result = try_load("input.wav");
std::visit([](auto&& val) {
    using T = std::decay_t<decltype(val)>;
    if constexpr (std::is_same_v<T, audio_buffer>)
        process(val);
    else
        std::cerr << "Erro: " << val << "\n";
}, result);
```

Exercícios Propostos

1. Implemente a classe `wav_loader` que usa `std::optional` para retornar o header WAV (struct `wav_header`) se válido, ou `nullopt` se o arquivo não for um WAV válido.

- 2.** Reescreva `load_wav` usando `std::variant<audio_buffer, file_error>` em vez de exceções. Compare as duas abordagens em termos de ergonomia e quando cada uma é mais apropriada.
- 3.** Implemente uma hierarquia de exceções para o Sintonia (`sintonia_error` → `file_error`, `format_error`, `processing_error`). Escreva testes Catch2 usando `REQUIRE_THROWS_AS` para cada tipo.
- 4.** Pesquise `std::expected` (C++23). Como ele melhora sobre `std::variant<T, E>` para tratamento de erros? Implemente `try_load` usando `std::expected` se seu compilador suportar.

Capítulo 22 — STL Panorâmica

🎯 Objetivos deste capítulo

- ▶ Conhecer os principais contêineres sequenciais e associativos.
- ▶ Usar algoritmos STL em vez de loops manuais.
- ▶ Compreender o modelo de iteradores.

22.1 Contêineres Essenciais

Contêiner	Caso de uso	Acesso	Inserção/remoção
vector<T>	Array dinâmico — padrão para sequências	O(1) random access	O(1) amortizado no final
deque<T>	Fila dupla — inserção/remoção eficiente nos dois extremos	O(1) random access	O(1) nos extremos
list<T>	Lista ligada — inserção/remoção O(1) em qualquer posição com iterador	O(n) sequencial	O(1) com iterador
map<K,V>	Mapa ordenado — busca O(log n)	O(log n)	O(log n)
unordered_map<K,V>	Mapa hash — busca O(1) amortizado	O(1) amortizado	O(1) amortizado
set<T>	Conjunto ordenado sem duplicatas	O(log n)	O(log n)
priority_queue<T>	Heap — acesso ao maior/menor em O(1)	O(1) no topo	O(log n)

22.2 Algoritmos STL

Algoritmos STL aplicados a audio

```
#include <algorithm>
#include <numeric>
#include <vector>

std::vector<float> v = {0.3f, -0.1f, 0.8f, 0.2f, -0.5f};

// Ordenar
std::sort(v.begin(), v.end());

// Máximo absoluto (peak level)
auto peak = *std::max_element(v.begin(), v.end(),
    [](float a, float b) { return std::abs(a) < std::abs(b); });
```

```
// RMS: raiz da média dos quadrados
float rms = std::sqrt(
    std::transform_reduce(v.begin(), v.end(), 0.0f,
        std::plus<>{}, [](float s) { return s * s; })
    / v.size());

// Remover silêncio (samples == 0)
auto new_end = std::remove_if(v.begin(), v.end(),
    [](float s) { return s == 0.0f; });
v.erase(new_end, v.end());
```

Capítulo 23 — Concorrência em C++

🎯 Objetivos deste capítulo

- ▶ Criar e gerenciar threads com `std::thread`.
- ▶ Proteger dados compartilhados com `std::mutex` e `std::lock_guard`.

23.1 `std::thread` — Criação e Join

`std::thread` — paralelismo básico

```
#include <thread>
#include <mutex>
#include <vector>

// Processamento paralelo de múltiplos buffers
void process_buffer(audio_buffer& buf, const effect& eff) {
    /* ... processamento ... */
}

std::vector<std::thread> workers;
for (auto& buf : buffers)
    workers.emplace_back(process_buffer, std::ref(buf), std::cref(eff));

for (auto& t : workers)
    t.join(); // aguarda todos terminarem
```

23.2 `std::mutex` e Race Conditions

`std::mutex` e `std::lock_guard`

```
std::mutex mtx;
int shared_counter = 0;

// Sem mutex: race condition – resultado indefinido
void unsafe() { ++shared_counter; } // NÃO thread-safe

// Com mutex: protege o acesso
void safe() {
    std::lock_guard<std::mutex> lock{mtx}; // RAII: libera ao sair do escopo
    ++shared_counter; // protegido
}
```

⚠ Cuidado:

Concorrência em C++ é complexa — race conditions, deadlocks e data races são bugs difíceis de reproduzir e depurar. Prefira usar tipos thread-safe da STL e padrões produtor-consumer a compartilhamento de estado mutável.

Capítulo 24 — Serialização

🎯 Objetivos deste capítulo

- ▶ Implementar serialização em texto puro.
- ▶ Usar nlohmann/json para serialização JSON.
- ▶ Compreender invariantes e versionamento de formato.

24.1 Estratégias de Serialização

Estratégia	Vantagens	Desvantagens
Texto puro	Simples, legível, depurável	Parsing manual, sem schema
JSON (nlohmann/json)	Legível, amplamente suportado, bibliotecas maduras	Verboso, sem tipos binários
Binário (Boost.Serialization)	Compacto, eficiente, versionamento	Não legível por humanos, dependência

24.2 Serialização JSON com nlohmann/json

Serialização JSON — projeto Sintonia

```
// CMakeLists.txt: FetchContent para nlohmann/json
FetchContent_Declare(json
  URL https://github.com/nlohmann/json/releases/download/v3.11.3/json.hpp
  DOWNLOAD_NO_EXTRACT TRUE)

// project.cpp
#include <nlohmann/json.hpp>
using json = nlohmann::json;

/* Serialização do projeto */
json project::to_json() const {
    json j;
    j["version"] = 1;
    j["name"] = name_;
    j["sample_rate"] = sample_rate_;
    j["tracks"] = json::array();
    for (const auto& t : tracks_)
        j["tracks"].push_back(t.to_json());
    return j;
}

/* Desserialização */
project project::from_json(const json& j) {
    if (j["version"] != 1)
        throw format_error{"versao de formato nao suportada"};
    project p{j["name"].get<std::string>(),
              j["sample_rate"].get<float>()};
    for (const auto& t : j["tracks"])
        p.add_track(track::from_json(t));
    return p;
}
```

Capítulo 25 — Princípios SOLID

🎯 Objetivos deste capítulo

- ▶ Identificar e aplicar cada um dos cinco princípios SOLID.
- ▶ Reconhecer violações de SOLID em código existente.
- ▶ Refatorar código para aderir aos princípios.
- ▶ Avaliar criticamente código gerado por LLMs sob a ótica SOLID.

25.1 S — Responsabilidade Única (SRP)

Uma classe deve ter apenas um motivo para mudar. Se uma classe gerencia a lógica de áudio E persiste dados E exibe UI, ela tem três motivos para mudar — qualquer alteração em um contexto pode quebrar os outros.

SRP — Responsabilidade Única

```
// ❌ VIOLA SRP: uma classe faz tudo
class audio_manager {
    void process_audio(audio_buffer&); // lógica de processamento
    void save_to_file(const std::string&); // persistência
    void update_ui(); // apresentação
};

// ✅ SRP: três classes, três responsabilidades
class audio_processor { void process(audio_buffer&); };
class audio_repository { void save(const project&, const std::string&); };
class audio_presenter { void update_display(const project&); };
```

25.2 O — Aberto/Fechado (OCP)

Classes devem estar abertas para extensão, mas fechadas para modificação. Adicionar um novo tipo de efeito não deve exigir modificar a classe effect_chain — ela deve aceitar qualquer effect por polimorfismo.

OCP — Aberto/Fechado via polimorfismo

```
// ❌ VIOLA OCP: adicionar distortion exige modificar process_chain
void process_chain(effect_type type, audio_buffer& buf) {
    if (type == GAIN) apply_gain(buf);
    else if (type == REVERB) apply_reverb(buf);
    // Adicionar DISTORTION: precisa modificar AQUI
}

// ✅ OCP: polimorfismo — adicionar tipo sem modificar código existente
void process_chain(std::vector<std::unique_ptr<effect>>& chain,
                  audio_buffer& buf) {
    for (auto& e : chain) e->process(buf);
    // Adicionar distortion_effect: não muda process_chain
}
```

25.3 L — Substituição de Liskov (LSP)

Objetos de uma subclasse devem poder substituir objetos da superclasse sem alterar o comportamento esperado do programa. Se uma function que aceita effect* não funciona corretamente com gain_effect*, há violação de LSP.

LSP — Substituição de Liskov

```
// ❌ VIOLA LSP: silent_effect ignora o buffer mas claims ser um effect
class silent_effect : public effect {
public:
    // Esperado: modificar buf. Resultado: buf inalterado. Surpresa!
    void process(audio_buffer&) override { /* faz nada */ }
};

// ✅ LSP: silent_effect tem semântica consistente — é um efeito de silêncio
class mute_effect : public effect {
public:
    void process(audio_buffer& buf) override {
        buf.apply([](float) { return 0.0f; }); // zera o buffer — efeito real
    }
};
```

25.4 I — Segregação de Interfaces (ISP)

Clientes não devem ser forçados a depender de interfaces que não usam. Uma interface grande que inclui funcionalidades irrelevantes para alguns clientes viola ISP.

ISP — Segregação de Interfaces

```
// ❌ VIOLA ISP: interface monolítica
class audio_component {
    virtual void process(audio_buffer&) = 0;
    virtual void save(const std::string&) = 0;
    virtual void display() = 0; // efeito de áudio não precisa de display!
};

// ✅ ISP: interfaces segregadas
class processable { virtual void process(audio_buffer&) = 0; virtual
~processable()=default; };
class serializable { virtual void save(const std::string&) = 0; virtual
~serializable()=default; };
class displayable { virtual void display() = 0; virtual
~displayable()=default; };

class gain_effect : public processable, public serializable { /* */ };
class ui_meter : public displayable { /* */ };
```

25.5 D — Inversão de Dependência (DIP)

Módulos de alto nível não devem depender de módulos de baixo nível — ambos devem depender de abstrações. Detalhes não devem depender de abstrações — abstrações não devem depender de detalhes.

DIP — Inversão de Dependência via injeção

```
// ❌ VIOLA DIP: audio_engine depende diretamente de arquivo
class audio_engine {
    file_repository repo_; // dependência concreta — difícil de testar
public:
    void save_project() { repo_.save_to_disk(/* ... */); }
};

// ✅ DIP: audio_engine depende de abstração
class project_repository {
public:
    virtual void save(const project&) = 0;
```



```
    virtual ~project_repository() = default;
};

class audio_engine {
    project_repository& repo_; // abstração – injeção de dependência
public:
    explicit audio_engine(project_repository& r) : repo_{r} {}
    void save_project(const project& p) { repo_.save(p); }
    // Testável: injete MockRepository nos testes
};
```

LLM:

Prompt para análise SOLID:

"Analisar o código abaixo e identifique violações dos cinco princípios SOLID. Para cada violação: nomeie o princípio, cite a linha problemática, explique o impacto (o que fica difícil de fazer) e mostre o código refatorado.",

[colar código a analisar]

Avalie a resposta: LLMs tendem a ser bons em SRP e OCP (padrões bem documentados), mas cometem erros em LSP (confundem com override correto) e DIP (confundem com qualquer uso de abstração).

Exercícios Propostos

1. Analise o código do Sintonia v1.0 (before SOLID) e identifique pelo menos 3 violações de SOLID com justificativas. Implemente as correções.
2. Refatore `audio_engine` para seguir DIP: crie a interface `project_repository`, implemente `file_repository` e `memory_repository` (para testes). Verifique que os testes não dependem de arquivo físico.
3. Identifique onde o Sintonia viola ISP: há alguma interface que alguns implementadores são forçados a implementar sem usar? Separe as interfaces.
4. Dada uma classe que viola SRP (ex.: faz parsing + validação + persistência), refatore em três classes colaborativas com responsabilidades únicas. Use injeção de dependência para conectá-las.

Capítulo 26 — Padrões de Projeto Canônicos em C++ Moderno

🎯 Objetivos deste capítulo

- ▶ Implementar Strategy, Observer, Factory Method em C++17.
- ▶ Implementar Decorator, Composite e Template Method.
- ▶ Aplicar Singleton corretamente (e reconhecer seus abusos).
- ▶ Avaliar quando cada padrão é apropriado vs. sobre-engenharia.

26.1 Strategy — Encapsular Algoritmos

Strategy define uma família de algoritmos intercambiáveis. No Sintonia, diferentes algoritmos de processamento de áudio (ganho, normalização, compressão) são strategies que podem ser trocadas em runtime.

Strategy — algorithms intercambiáveis

```
// Strategy: algoritmo de processamento intercambiável
class processing_strategy {
public:
    virtual void apply(audio_buffer& buf) = 0;
    virtual ~processing_strategy() = default;
};

class normalize_strategy : public processing_strategy {
    void apply(audio_buffer& buf) override { /* normaliza */ }
};

class compress_strategy : public processing_strategy {
    float threshold_;
public:
    explicit compress_strategy(float t) : threshold_{t} {}
    void apply(audio_buffer& buf) override { /* compressão dinâmica */ }
};

// Context: usa qualquer strategy
class audio_processor {
    std::unique_ptr<processing_strategy> strategy_;
public:
    void set_strategy(std::unique_ptr<processing_strategy> s) {
        strategy_ = std::move(s);
    }
    void process(audio_buffer& buf) {
        if (strategy_) strategy_->apply(buf);
    }
};
```

26.2 Observer — Notificação de Eventos

Observer — transport notifica observadores

```
class transport_observer {
public:
    virtual void on_play() = 0;
    virtual void on_stop() = 0;
    virtual void on_record() = 0;
    virtual ~transport_observer() = default;
};
```

```

class transport {
    std::vector<transport_observer*> observers_;
public:
    void add_observer(transport_observer* o) { observers_.push_back(o); }
    void play() { for (auto* o : observers_) o->on_play(); }
    void stop() { for (auto* o : observers_) o->on_stop(); }
    void record() { for (auto* o : observers_) o->on_record(); }
};

class ui_meter : public transport_observer {
public:
    void on_play() override { std::cout << "[UI] Playing\n"; }
    void on_stop() override { std::cout << "[UI] Stopped\n"; }
    void on_record() override { std::cout << "[UI] Recording\n"; }
};

```

26.3 Factory Method — Criação Polimórfica

Factory Method — criação sem expor hierarquia

```

// Factory Method: cria o tipo correto sem expor a hierarquia
class effect_factory {
public:
    static std::unique_ptr<effect> create(const std::string& type,
                                         const json& params) {
        if (type == "gain") return std::make_unique<gain_effect>(
            params["factor"].get<float>());
        if (type == "reverb") return std::make_unique<reverb_effect>(
            params["room_size"].get<float>());
        throw format_error{"tipo de efeito desconhecido: " + type};
    }
};

// Uso: deserializa cadeia de efeitos do JSON sem switch/case no cliente
for (const auto& eff_json : project_json["effects"])
    chain.push_back(effect_factory::create(
        eff_json["type"], eff_json["params"]));

```

26.4 Decorator — Efeitos Encadeados

Decorator adiciona responsabilidades a objetos dinamicamente, sem subclasse. Cada decorator envolve outro effect e adiciona comportamento antes ou depois de process().

Decorator — adiciona comportamento sem modificar

```

// Decorator base: delega para o effect envolvido
class effect_decorator : public effect {
    std::unique_ptr<effect> inner_;
protected:
    effect& inner() { return *inner_; }
public:
    explicit effect_decorator(std::unique_ptr<effect> inner)
        : effect{inner->name()}, inner_{std::move(inner)} {}
    void process(audio_buffer& buf) override { inner_->process(buf); }
};

// Decorator concreto: adiciona logging ao effect
class logging_decorator : public effect_decorator {
public:
    using effect_decorator::effect_decorator;
};

```

```

    void process(audio_buffer& buf) override {
        std::cout << "[LOG] Antes de " << name() << "\n";
        effect_decorator::process(buf);
        std::cout << "[LOG] Depois de " << name() << "\n";
    }
};

// Uso: adiciona logging sem modificar gain_effect
auto logged_gain = std::make_unique<logging_decorator>(
    std::make_unique<gain_effect>(0.5f));

```

26.5 Composite — Hierarquia de Efeitos

Composite — group de efeitos como efeito único

```

// Composite: effect_group é tratado como um effect único
class effect_group : public effect {
    std::vector<std::unique_ptr<effect>> effects_;
public:
    explicit effect_group(std::string name) : effect{std::move(name)} {}
    void add(std::unique_ptr<effect> e) { effects_.push_back(std::move(e)); }
    void process(audio_buffer& buf) override {
        for (auto& e : effects_) e->process(buf);
    }
};

// Uso: group de efeitos como se fosse um único efeito
auto mastering = std::make_unique<effect_group>("mastering");
mastering->add(std::make_unique<compress_effect>(0.7f));
mastering->add(std::make_unique<limiter_effect>(0.9f));
// mastering pode ser colocado em qualquer effect_chain

```

26.6 Singleton — Uso Correto e Abusos

Singleton de Meyers — implementação correta

```

// Singleton de Meyers: thread-safe em C++11, sem new/delete
class audio_config {
public:
    static audio_config& instance() {
        static audio_config inst; // inicializado uma vez, thread-safe
        return inst;
    }

    float master_volume() const { return master_volume_; }
    void set_master_volume(float v) { master_volume_ = v; }

    audio_config(const audio_config&) = delete;
    audio_config& operator=(const audio_config&) = delete;

private:
    audio_config() = default;
    float master_volume_ = 1.0f;
};

```

⚠ Cuidado:

Singleton é o padrão mais frequentemente abusado. Use-o apenas quando há genuinamente uma única instância com estado global necessário (configuração do

sistema, logger). Singleton torna o código difícil de testar (estado compartilhado entre testes) e cria acoplamento implícito. Prefira injeção de dependência.

Exercícios Propostos

1. Implemente o padrão Template Method para um pipeline de processamento de áudio: a classe base define a sequência (carrega → normaliza → processa → salva) mas delega cada passo para subclasses.
2. Aplique Decorator para adicionar um metering_decorator que mede o nível de pico (peak level) antes e depois de cada efeito sem modificar os efeitos existentes.
3. Refatore a cadeia de efeitos do Sintonia v2.3 aplicando: Composite para grupos de efeitos, Strategy para seleção de algoritmo de compressão, Observer para notificar a UI quando a cadeia é modificada.
4. Identifique 3 casos de uso de Singleton que são abusos (onde DIP seria melhor). Para cada um, mostre como refatorar usando injeção de dependência.

Capítulo 27 — Qt na Prática e LLMs no Ciclo OO

🎯 Objetivos deste capítulo

- ▶ Compreender a arquitetura do Qt: QObject, signals/slots, event loop.
- ▶ Criar uma janela básica com botões e callbacks.
- ▶ Conectar domínio e apresentação via signals/slots.
- ▶ Aprofundar o uso crítico de LLMs no ciclo OO completo.

27.1 Arquitetura do Qt

Qt é um framework C++ para interfaces gráficas, redes, bancos de dados e muito mais. O mecanismo central é signals/slots — uma forma de comunicação frouxa entre objetos que implementa o padrão Observer sem depender de virtual. QObject é a classe raiz de todos os objetos Qt; o event loop (QApplication::exec()) processa eventos de interface e dispara slots.

Conceito Qt	Equivalente OO clássico
Signal	Notificação de evento — análogo ao método "notify" do Observer
Slot	Handler de evento — análogo ao "update" do Observer
connect()	Registrar observer — Liga um signal a um slot, sem acoplamento direto
QObject	Classe base com metadados (MOC) para signals/slots e propriedades
QApplication::exec()	Event loop — processa eventos de UI e dispara slots

mixer_window.cpp — janela básica do Sintonia Qt

```
// Exemplo mínimo: janela com botão Play/Stop para o Sintonia
#include <QApplication>
#include <QMainWindow>
#include <QPushButton>
#include <QVBoxLayout>

class mixer_window : public QMainWindow {
    Q_OBJECT // macro obrigatória para signals/slots
public:
    explicit mixer_window(QWidget* parent = nullptr)
        : QMainWindow{parent}
    {
        auto* central = new QWidget{this};
        auto* layout = new QVBoxLayout{central};

        auto* btn_play = new QPushButton{"▶ Play", central};
        auto* btn_stop = new QPushButton{"■ Stop", central};
        layout->addWidget(btn_play);
        layout->addWidget(btn_stop);
        setCentralWidget(central);

        /* connect(emissor, signal, receptor, slot) */
        connect(btn_play, &QPushButton::clicked, this, &mixer_window::on_play);
        connect(btn_stop, &QPushButton::clicked, this, &mixer_window::on_stop);
    }
}
```

```

signals:
    void play_requested();
    void stop_requested();

private slots:
    void on_play() { emit play_requested(); }
    void on_stop() { emit stop_requested(); }
};

int main(int argc, char* argv[]) {
    QApplication app{argc, argv};
    mixer_window win;
    win.show();
    return QApplication::exec();
}

```

27.2 Separação Domínio/Apresentação com Qt

A camada de domínio (audio_engine, project, effect_chain) não deve depender do Qt. A camada de apresentação (mixer_window) conecta-se ao domínio via signals/slots — quando o usuário clica "Play", a UI emite um signal que é recebido pelo audio_engine sem que a UI saiba da implementação de áudio.

DIAGRAMA UML

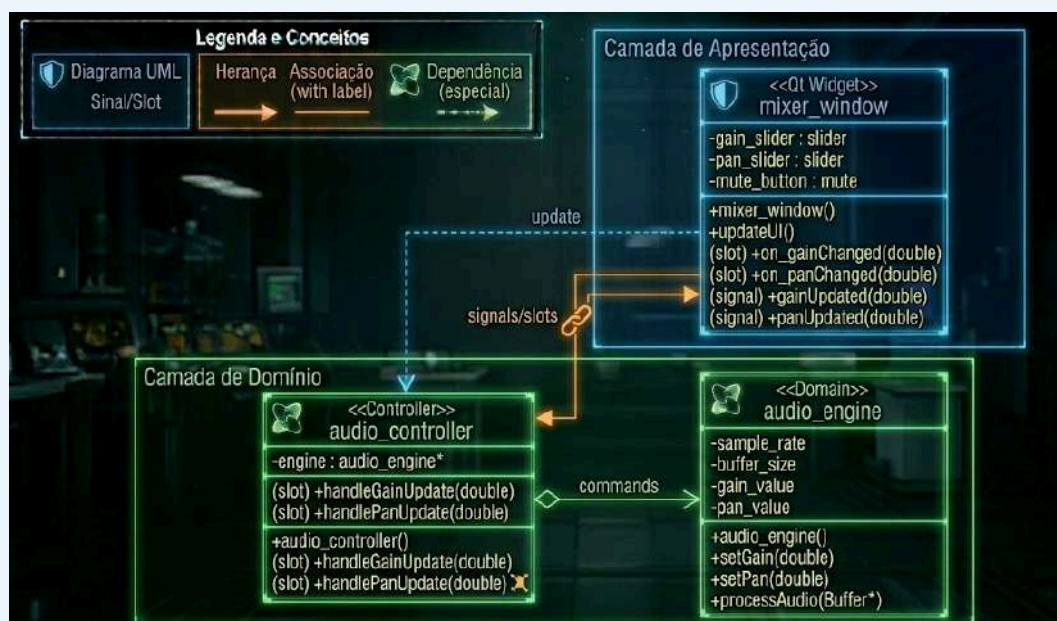


Diagrama de classes: mixer_window (Qt) conectada a audio_controller via signals/slots. audio_controller usa audio_engine (domínio). Separação clara entre camadas de apresentação e domínio.

27.3 LLMs no Ciclo OO — Aprofundamento

1. Após estudar toda a disciplina, você está em posição de usar LLMs de forma verdadeiramente crítica e produtiva. O ciclo de desenvolvimento OO assistido por LLM envolve:

2. Modelagem: "Dado o domínio X, gere um diagrama UML em Mermaid identificando classes, responsabilidades e relacionamentos. Aplique SRP e OCP."
3. Implementação de esqueleto: "Gere a declaração C++17 das classes do diagrama, com construtores, destrutor virtual na base, smart pointers para dependências."
4. Geração de testes: "Para a classe `audio_buffer`, gere testes Catch2 cobrindo construção, cópia, movimento, acesso válido e inválido, e após `clear()`."
5. Refatoração: "Este código viola SOLID desta forma: [explicar]. Refatore aplicando o princípio correto, mantendo a interface pública existente."
6. Revisão crítica: "Revise o código abaixo identificando: (a) vazamentos de memória; (b) violações da regra dos cinco; (c) race conditions; (d) violações de SOLID."
7. Documentação: "Gere comentários Doxygen para a classe abaixo, documentando precondições, pós-condições e invariantes."

LLM:

Análise crítica — o que LLMs continuam errando em C++ mesmo com bons prompts:

1. Esquecem `noexcept` nos construtores de movimento.
2. Geram código com `new/delete` cru mesmo quando instruídos a usar smart pointers.
3. Implementam a regra dos três mas esquecem as operações de movimento (regra dos cinco).
4. Geram tipos-valor onde tipos-polimórficos seriam corretos (e vice-versa).
5. Ignoram thread-safety em singletons ou dados compartilhados.
6. Criam dependências circulares de headers sem perceber.

Para cada ponto: verifique explicitamente no código gerado antes de usar.

Exercícios Propostos

1. Crie a janela `mixer_window` completa para o Sintonia v2.4: botões Play/Stop/Record, um slider de volume, e um display de nível de pico. Conecte os controles ao `audio_engine` via `signals/slots`.
2. Separe rigorosamente a camada de domínio da camada Qt: o `audio_engine` não deve incluir nenhum header Qt. Demonstre que o `audio_engine` pode ser testado com Catch2 sem inicializar o `QApplication`.
3. Use um LLM para gerar o código completo do Sintonia v2.4. Analise criticamente o resultado: (a) segue DIP? (b) a UI depende do domínio ou o contrário? (c) há violações SOLID? (d) os smart pointers estão corretos?
4. Escreva um documento de post-mortem do projeto Sintonia: o que foi mais difícil de implementar, quais decisões de design (herança vs. composição, virtual vs. template, etc.) você mudaria com o conhecimento adquirido, e como os LLMs ajudaram e atrapalharam no processo.

Glossário

Termos fundamentais do material, em ordem alfabética.

Abstração: Mecanismo pelo qual detalhes de implementação são ocultados, expondo apenas o essencial para o usuário de uma classe ou módulo.

AddressSanitizer (ASan): Instrumentação do compilador que detecta em runtime erros de memória: uso após liberação (use-after-free), acesso fora dos limites, vazamentos.

auto: Palavra-chave de C++11 que instrui o compilador a deduzir o tipo de uma variável a partir da expressão de inicialização.

Classe abstrata: Classe com pelo menos uma função virtual pura (= 0); não pode ser instanciada diretamente.

CMake: Sistema de build multiplataforma para projetos C++; gera Makefiles ou projetos IDE a partir de CMakeLists.txt.

Composição: Relação "tem um" (has-a) entre classes; alternativa à herança quando não há relação is-a.

Concept (C++20): Conjunto de restrições sobre parâmetros de template, verificadas em tempo de compilação com mensagens de erro legíveis.

const-correctness: Prática de marcar como const todos os métodos, parâmetros e variáveis que não devem modificar dados; hábito fundamental em C++.

CRTP: Curiously Recurring Template Pattern — técnica de polimorfismo estático onde uma classe herda de template parametrizado por ela mesma.

Despacho dinâmico: Resolução em tempo de execução de qual implementação de um método virtual chamar, via vtable.

Destrutor virtual: Destrutor declarado virtual em classe base; garante que o destrutor da classe derivada seja chamado ao deletar via ponteiro base.

DIP: Dependency Inversion Principle — módulos de alto nível dependem de abstrações, não de implementações concretas.

dynamic_cast: Operador de conversão de tipo verificado em runtime; retorna nullptr (ponteiro) ou lança bad_cast (referência) se o tipo for incompatível.

Encapsulamento: Agrupamento de dados e operações em uma classe, com controle de acesso (public/protected/private).

event loop: Loop que processa eventos de interface (cliques, teclas, rede) e dispara handlers/slots; núcleo de aplicações Qt.

Factory Method: Padrão de projeto que define uma interface para criação de objetos, permitindo que subclasses decidam qual classe instanciar.

GDB: GNU Debugger — ferramenta para depurar programas C++: breakpoints, step, inspect, backtrace.

Herança múltipla: Capacidade de uma classe derivar de mais de uma classe base; C++ suporta; Java e C# restringem a interfaces.

Herança virtual: Mecanismo de C++ para resolver o problema do diamante: garante que a classe base compartilhada tenha apenas uma instância na hierarquia.

ISP: Interface Segregation Principle — interfaces pequenas e específicas; clientes não devem depender de métodos que não usam.

lvalue: Expressão com endereço de memória acessível; pode aparecer à esquerda de uma atribuição — variáveis nomeadas.

LLM: Large Language Model — modelo de linguagem de grande escala treinado em texto e código; usado como assistente de programação.

LSP: Liskov Substitution Principle — objetos de uma subclasse devem poder substituir a superclasse sem alterar o comportamento correto do programa.

Move semantics: Mecanismo de C++11 para transferir recursos de um objeto para outro sem copiar; implementado via construtor e operador de atribuição de movimento.

noexcept: Especificador que garante que uma função não lança exceções; permite otimizações pelo compilador e pela STL (ex.: realloc de vector).

nullptr: Literal de tipo `nullptr_t` introduzido em C++11 para representar ponteiro nulo, substituindo NULL (que é apenas 0 inteiro).

OCP: Open/Closed Principle — classes abertas para extensão (novos tipos), fechadas para modificação (sem alterar código existente).

Observer: Padrão de projeto onde objetos (observers) são notificados automaticamente quando o estado de outro objeto (subject) muda.

override: Especificador de C++11 que garante em tempo de compilação que uma função realmente sobrescreve um virtual da classe base.

Polimorfismo: Capacidade de objetos de tipos diferentes responderem à mesma mensagem de maneiras distintas.

RAII: Resource Acquisition Is Initialization — idioma C++ onde recursos são adquiridos no construtor e liberados no destrutor.

Ranges (C++20): Biblioteca que generaliza algoritmos STL para trabalhar com views lazy e composição funcional de transformações.

Regra dos Cinco: Se uma classe declara destrutor, construtor de cópia ou atribuição de cópia, deve declarar todos os cinco especiais (incluindo movimento).

Regra do Zero: Classe que usa apenas membros com semântica de valor correta (vector, string, unique_ptr) não precisa declarar nenhuma operação especial.

shared_ptr: Smart pointer de propriedade compartilhada com contagem de referências; libera o objeto quando a última cópia é destruída.

Singleton: Padrão de projeto que garante uma única instância de uma classe com ponto de acesso global; usar com cautela (dificulta testes).

SOLID: Cinco princípios de projeto OO: SRP, OCP, LSP, ISP, DIP; guiam código coeso, extensível e testável.

SRP: Single Responsibility Principle — uma classe deve ter apenas um motivo para mudar.

std::optional: Tipo de C++17 que representa um valor que pode ou não estar presente; alternativa a ponteiros nulos e exceções para resultados opcionais.

std::variant: Tipo de C++17 que armazena exatamente um de vários tipos possíveis; útil para modelar resultado ou erro sem exceções.

Strategy: Padrão de projeto que encapsula algoritmos intercambiáveis dentro de objetos, permitindo seleção em runtime.

Template: Mecanismo de C++ para escrita de código genérico parametrizado por tipo; instanciado pelo compilador para cada tipo concreto usado.

this: Ponteiro implícito dentro de todo método não-estático de C++; aponta para o objeto sobre o qual o método foi chamado.

typeid: Operador RTTI que retorna informação de tipo em runtime (type_info); requer que a classe tenha pelo menos uma função virtual.

UBSan: UndefinedBehaviorSanitizer — instrumentação que detecta comportamento indefinido em runtime: overflow de inteiro, desreferência de nullptr, etc.

unique_ptr: Smart pointer de propriedade exclusiva; não pode ser copiado, apenas movido; sem overhead em relação ao raw pointer.

UML: Unified Modeling Language — notação gráfica padrão para modelagem de sistemas de software; usada nesta disciplina em diagramas de classes e sequência.

virtual: Palavra-chave que indica que um método pode ser sobrescrito em classes derivadas e é resolvido em tempo de execução via vtable.

vtable: Tabela de ponteiros para funções virtuais, gerada pelo compilador para cada classe polimórfica; cada objeto tem um vptr apontando para ela.

weak_ptr: Smart pointer observador; não incrementa a contagem de referências do shared_ptr; usado para quebrar ciclos de referência.

Referências Bibliográficas

Referências Básicas

1. STROUSTRUP, Bjarne. A Tour of C++. 3. ed. Boston: Addison-Wesley, 2022.
2. MEYERS, Scott. C++ Moderno e Eficaz: 42 formas específicas de aprimorar seu uso de C++11 e C++14. Rio de Janeiro: Alta Books, 2015.
3. DEITEL, Harvey M.; DEITEL, Paul J. C++: como programar. São Paulo: Pearson Prentice Hall. (Edição mais recente disponível no acervo BC/UFPB.)
4. GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. Padrões de projeto: soluções reutilizáveis de software orientados a objetos. Porto Alegre: Bookman, 2000.
5. MENDES, Antonio. Introdução à programação orientada a objetos com C++. Rio de Janeiro: Campus/Elsevier.

Referências Complementares

1. STROUSTRUP, Bjarne. The C++ Programming Language. 4. ed. Boston: Addison-Wesley, 2013.
2. MARTIN, Robert C. Código limpo: habilidades práticas do Agile Software. Rio de Janeiro: Alta Books, 2009.
3. MARTIN, Robert C. Arquitetura limpa: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019.
4. MEYERS, Scott. C++ eficaz: 55 maneiras de aprimorar seus programas e projetos. 3. ed. Porto Alegre: Bookman, 2011.
5. JOSUTTIS, Nicolai M. The C++ Standard Library: a tutorial and reference. 2. ed. Boston: Addison-Wesley, 2012.
6. FOWLER, Martin. UML essencial: um breve guia para a linguagem-padrão de modelagem de objetos. 3. ed. Porto Alegre: Bookman, 2005.
7. STROUSTRUP, Bjarne; SUTTER, Herb (eds.). C++ Core Guidelines. Disponível em: <https://isocpp.github.io/CppCoreGuidelines>
8. cppreference.com. Referência completa do C++ moderno. Disponível em: <https://en.cppreference.com>
9. THE QT COMPANY. Qt Documentation. Disponível em: <https://doc.qt.io>
10. Catch2 Framework. Disponível em: <https://github.com/catchorg/Catch2>
11. nlohmann/json. Disponível em: <https://github.com/nlohmann/json>